EdgeSign: An IoT-Enabled Edge–Cloud Hybrid System for Real-Time Sign Language Translation

A PROJECT REPORT

Submitted by

Jeevan Baabu Murugan RA2211026010548
Surya Sathyanarayanan RA2211026010562

Under the Guidance of

Dr. R.Babu

Associate Professor,
Department of Computational Intelligence

in partial fulfillment of the requirementsfor the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in Artificial Intelligence And Machine learning

DEPARTMENT OF COMPUTATIONAL INTELLIGENCE COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE ANDTECHNOLOGY

KATTANKULATHUR- 603 203

MAY 2026

Department of Computational Intelligence
SRM Institute of Science & Technology
Own Work Declaration Form

This sheet must be filled in (each box ticked to show that the condition has been met). It must be signed and dated along with your student registration number and included with all assignments you submit – work will not be marked unless this is done.

To be completed by the student for all assessments

Degree/ Course	**: B.Tech CSE AIML**
Student Name	**: Jeevan Babu Murugan, Surya Sathyanarayanan**
Registration Number	**:RA2211026010548, RA2211026010562**
Title of Work	**: Edge Sign: An IOT-Enabled Edge Cloud Hybrid System For Real Time Sign Language translation System**

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is my / our own except where indicated, and that We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that we have received from others (e.g.fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

We understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

We are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is my / our own work, except where indicated by referring, and that we have followed the good academic practices noted above.

SURYA SATHYANARAYANAN

JEEVAN BABU

If you are working in a group, please write your registration numbers and sign with the date for every student in your group.

SRM

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSP401L - Project report titled **EdgeSign: An IoT-Enabled Edge–Cloud Hybrid System for Real-Time Sign Language Translation** is the Bonafide work of **Jeevan Baabu Murugan [RA2211026010548], Surya Sathyanarayanan [RA2211026010562]** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr.R. BABU

SUPERVISOR
Associate Professor
DEPARTMENT OF
COMPUTATIONAL
INTELLIGENCE

SIGNATURE

DR. R ANNIE UTHRA

PROFESSOR &HEAD
OF THE DEPARTMENT
DEPARTMENT OF
COMPUTATIONAL
INTELLIGENCE

EXAMINER 1

EXAMINER 2

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr. C Lakshmi,** Professor and Associate Chairperson-AI, School of Computing, SRM Institute of Science and Technology, for her invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. R Annie Uthra** , Professor and Head Department Of Computational Intelligence, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head, and Panel Members Department of Computational Intelligence, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr. Kanipriya M**, Department of Computational Intelligence, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr.R. Babu** , Department of Computational Intelligence, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under his mentorship. He provided us with the freedom and support to explore the research topics of our interest. His passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of CINTEL, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

Authors

SURYA SATHYANARAYANAN [RA2211026010562]

JEEVAN BAABU MURUGAN [RA22110206010548]

ABSTRACT

Communication barriers faced by individuals with speech and hearing impairments often limit their ability to interact effectively in everyday situations. This project presents EdgeSign, an IoT-enabled edge–cloud hybrid system designed for real-time sign language translation into text and speech. The system utilizes a Raspberry Pi 4–based edge device equipped with a camera to capture hand gestures, which are processed using the MediaPipe framework for hand landmark extraction. A machine learning model, specifically a Random Forest classifier, is employed to accurately recognize predefined gestures with low latency. The recognized gestures are converted into text and further processed to generate meaningful sentences, enhancing communication clarity. A Text-to-Speech (TTS) module is integrated to produce audible output, enabling verbal communication with non-sign language users. Additionally, the system supports real-time interaction through a mobile application, where recognized text is displayed and user customization options are provided via WebSocket or BLE communication. Cloud integration is incorporated to improve sentence formation and enable scalable processing. The proposed system emphasizes portability, low latency, and real-world usability, making it suitable for deployment in environments such as healthcare centers, educational institutions, and public spaces. Experimental results demonstrate reliable gesture recognition and efficient system performance under standard conditions. Overall, EdgeSign provides an effective, accessible, and scalable solution for bridging communication gaps and promoting inclusive interaction for individuals with communication disabilities. The system also incorporates additional features such as OCR-based text extraction, speech-to-text input, and a developer mode for previewing gesture detection and system logs. These features enhance system flexibility, enable multimodal interaction, and support continuous improvement through data collection and monitoring, making the solution more robust and adaptable for real-world deployment. Furthermore, the inclusion of debugging tools helps in analyzing system performance and identifying potential errors during operation. The modular design of the system allows easy integration of new features and future upgrades. This ensures that the system remains scalable and can evolve with advancements in technology and user requirements.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

BLE	Bluetooth Low Energy
DFD	Data Flow Diagram
ID	Identity Document (used in context of "ID card with camera")
LSTM	Long Short-Term Memory
NLP	Natural Language Processing
SDG	Sustainable Development Goal
SVM	Support Vector Machine
TTS	Text-to-Speech
UI	User Interface
US	User Story
DFD	Data Flow Diagram
TTS	Text-To-Speech
SVOX Pico	Small Voice Engine (TTS engine)
SQLite	Structured Query Language Lite (embedded DB)
Pi	Raspberry Pi
Flask	Python Web Framework (used implicitly but not abbreviated)

CHAPTER 1

INTRODUCTION

1.1 Introduction to AI-Powered Edge-Sign Language Translator

Advancements in artificial intelligence, computer vision, and embedded systems have enabled the development of intelligent assistive technologies that improve human–computer interaction. However, communication remains a significant challenge for individuals with speech and hearing impairments, particularly when interacting with people who are not familiar with sign language. This communication gap often limits their participation in everyday activities such as education, work, and social interaction.

Sign language is an effective mode of communication for such individuals, but its limited understanding among the general population creates a barrier. Existing solutions, such as human interpreters or text-based communication, are not always practical or available in real-time scenarios. Although several gesture recognition systems have been developed, many rely on high computational resources or function only in controlled environments, making them less suitable for real-world applications.

To address these challenges, this project proposes EdgeSign: An IoT-Enabled Edge–Cloud Hybrid System for Real-Time Sign Language Translation. The system captures hand gestures using a camera connected to a Raspberry Pi and extracts hand landmarks using the MediaPipe framework. These landmarks are processed using a machine learning model to recognize predefined gestures. The recognized data is then transmitted to the cloud, where advanced processing, including sentence generation and contextual enhancement, is performed. Finally, the output is converted into speech using a Text-to-Speech (TTS) module.

The proposed system combines edge and cloud computing to achieve low latency and improved performance. Edge processing enables fast gesture detection, while cloud integration enhances accuracy and scalability. The system is designed to be portable, efficient, and suitable for real-time use in practical environments. Overall, EdgeSign aims to provide an accessible and reliable solution that bridges the communication gap and supports inclusive interaction for individuals with speech and hearing impairments.

1.2 Motivation

Although significant progress has been made in computer vision and machine learning, many gesture recognition systems are still limited in terms of real-world usability. Most existing approaches rely on high-end hardware, controlled environments, or continuous cloud connectivity, which makes them less practical for everyday use. These limitations highlight the need for a solution that is both efficient and deployable on low-cost, portable devices without compromising performance.

Another key challenge lies in achieving real-time response. Systems that depend entirely on cloud processing often introduce latency due to network delays, while fully edge-based systems may struggle with limited computational resources. This creates a trade-off between speed and accuracy that must be carefully balanced. Therefore, there is a strong motivation to design a hybrid system that combines the advantages of edge computing for fast data processing and cloud computing for advanced analysis and contextual understanding.

In addition, many existing systems focus only on recognizing individual gestures, without converting them into meaningful sentences or natural speech. This limits their effectiveness in real communication scenarios, where context and clarity are important. Integrating conversational AI and speech generation can significantly improve the usability and impact of such systems.

Furthermore, there is a growing need for assistive technologies that are affordable, scalable, and easy to use. By leveraging devices like Raspberry Pi and lightweight machine learning models, it becomes possible to develop a cost-effective solution that can be widely adopted. This motivates the development of the EdgeSign system, which aims to provide a practical, real-time, and intelligent communication aid by combining gesture recognition, edge–cloud processing, and speech synthesis in a single integrated framework.

Another motivating factor is the need for continuous improvement and adaptability in gesture recognition systems. Users may have variations in gesture style, speed, and hand orientation, which can affect system accuracy if not properly handled. Therefore, a system that can be easily updated with new data and improved models is essential for long-term usability.

1.3 Sustainable Development Goal of the Project

The proposed EdgeSign system aligns with the Sustainable Development Goal (SDG) 10: Reduced Inequalities, which focuses on promoting social, economic, and political inclusion for all individuals. Communication barriers faced by individuals with speech and hearing impairments often limit their participation in education, employment, and social interactions. By providing a real-time sign language translation system, this project aims to bridge this gap and enable more inclusive communication.

The system enhances accessibility by allowing users to convert sign language gestures into text and speech, making it easier for them to interact with people who are not familiar with sign language. This contributes to reducing inequalities by ensuring that differently-abled individuals have equal opportunities to express themselves and engage in daily activities. The use of affordable hardware such as Raspberry Pi also supports accessibility by making the solution cost-effective and scalable.

In addition, the project promotes digital inclusion by leveraging modern technologies such as edge computing, cloud integration, and artificial intelligence. These technologies help create an efficient and user-friendly assistive system that can be deployed in various environments, including schools, workplaces, and public spaces. Overall, the EdgeSign system contributes toward building a more inclusive society by empowering individuals with communication challenges and supporting the objectives of sustainable development.

Furthermore, the project also indirectly supports SDG 4: Quality Education by enabling better communication in learning environments for students with speech and hearing impairments. By providing a tool that translates sign language into understandable text and speech, the system helps create a more inclusive classroom setting where students can actively participate and interact with teachers and peers. This not only improves learning outcomes but also encourages equal educational opportunities. The integration of assistive technology in education further promotes awareness and acceptance, contributing to a more inclusive and equitable society.

1.4 Product Vision Statement

1.4.1 Audience

Primary Audience:

- Individuals with speech or hearing impairments who require an intuitive, accessible, and real-time communication assistant to bridge the gap between sign language and spoken language.

Secondary Audience:

- Caregivers, teachers, therapists, and family members who support individuals with communication impairments and require effective tools to improve interaction with people.
- Inclusive organizations such as old Age-house,schools, hospitals, rehabilitation centers, and community service institutions that promote accessibility and assistive communication tech

1.4.2 Needs

Primary Needs:

- Real-time translator from sign language to spoken natural language by using a ML model.
- Accurate and low-latency gesture recognition using lightweight machine learning models. Portable and cost-effective system that can be used in everyday environments.

Secondary Needs:

- Cloud integration for advanced processing and contextual sentence generation by API key
- Wireless audio output through speakers or headphones for seamless realtime communication.
- User-friendly interface for configuring gestures and monitoring system performance by Dev.

1.4.3 Products

Core Product:

- EdgeSign system: An edge–cloud hybrid sign language translation system using Raspberry Pi, MediaPipe for hand tracking, and machine learning models (Random Forest / ML model).

Additional Features:

- AI-based sentence generation using cloud integration (Gemini or similar) for generation.
- Text-to-Speech (TTS) module for real-time voice output (sentence generation it's converted TTS).
- WebSocket-based communication enables efficient real-time data exchange between the edge device.
- Dataset logging and storage support continuous machine learning model improvement, performance.

1.4.4 Values

Core Values:

- Accessibility: Ensures communication is easy, inclusive, and available to everyone regardless of speech or hearing limitations.
- Efficiency: Delivers real-time system performance with minimal latency for smooth and responsive communication.
- Portability: enables the system to operate on compact, lightweight, and low-cost devices such as Raspberry Pi for everyday convenience.
- Scalability: Supports cloud integration for advanced processing capabilities, system enhancement, and future technological expansion.
- Usability: Provides a simple, intuitive, and user-friendly interaction experience suitable for daily use.
- Future Scope**:** Can be extended with improved models, larger datasets, and enhanced real-world deployment.

The EdgeSign system is designed to provide an accessible, efficient, and user-friendly communication solution for individuals with speech and hearing impairments. It aims to bridge communication gaps by translating sign language gestures into meaningful text and speech in real time using a compact and portable edge device based on Raspberry Pi 4. The system leverages MediaPipe for accurate hand landmark detection and a machine learning model, such as Random Forest, for reliable gesture classification with low latency. By integrating edge–cloud architecture, the system ensures fast local processing while enabling advanced sentence generation and contextual understanding through cloud support. Additional features such as mobile application integration, wireless audio output, gesture customization, and real-time feedback enhance usability and adaptability in various environments. Designed with a focus on scalability, portability, and real-world deployment, the system can be effectively used in healthcare centers, educational institutions, workplaces, and home settings. Overall, EdgeSign promotes independent communication, improves accessibility, and supports inclusive interaction, making it a practical assistive technology solution for everyday use.

The EdgeSign system focuses on delivering a simple, reliable, and efficient communication experience for users by combining gesture recognition, text generation, and speech output into a unified platform. Its design emphasizes ease of use, portability, and real-time performance .

1.5 Product Goal

By offering a real-time sign language translation solution, the suggested system aims mostly to improve communication access for people with hearing and speech impairments. The system transforms hand signals into structured text and vocal output in order to allow easy and free communication in daily living. The system provides a straight, effective, and user-friendly communication route by lowering dependence on human interpreters and conventional means of communication.

Using the MediaPipe framework, which extracts hand markers in real time, gesture tracking is done. An algorithm based on machine learning, such as a Random Forest classifier, is used to precisely identify prearranged motions by means of these markers. The accepted gestures are next sent across an edge–cloud communication channel, where sophisticated processing produces pertinent and context-aware statements. This guarantees that the result is not confined to individual words but creates whole and natural sentences fit for actual interaction.

Furthermore included in the system are capabilities like gesture-to-text translation, text-to-speech (TTS) production, and cloud-based sentence enhancement. Speakers or Bluetooth-enabled devices supply audio output, which gives versatility in several application settings. Moreover, the system supports data logging and customisation options so that gesture recognition and adaptability can be constantly enhanced in line with user requirements.

Through the combination of edge and cloud computing, the project aims at low latency and high efficiency. While cloud processing improves accuracy and contextual understanding, edge processing helps quick gesture recognition and initial categorization. Making it ideal for use in public areas, healthcare facilities, and educational institutions, the system is meant to be scalable, affordable, and portable.

The objective of the product is ultimately to develop an intelligent assistive communication platform that encourages inclusivity, freedom, and accessibility. The system enables users to express themselves more clearly and engage boldly in everyday interactions by bridging sign language with spoken communication.

1.6 Product Backlog

The following table 1.1 represents the user stories in the form of a backlog table of the edge sign translation.

Table 1.1 Product backlog table

S.No	User Stories
#US 1	As a user, I want the system to recognize my hand gestures accurately so that I can communicate effectively.
#US 2	As a user, I want my gestures to be converted into meaningful text so that I can form sentences.
#US 3	As a user, I want the system to generate speech output so that others can understand my message.
#US 4	As a user, I want the system to send gesture data to the cloud for improved processing and sentence generation.
#US 5	As a user, I want real-time response with minimal delay so that communication feels natural.
#US 6	As a user, I want to add custom gestures so that I can personalize communication.
#US 7	As a user, I want the system to ignore unknown gestures to avoid incorrect outputs.
#US 8	As a user, I want to see recognized words or sentences on a display or mobile interface.
#US 9	As a user, I want OCR-based text recognition so that I can capture and understand written text.
#US 10	As a user, I want to enable or disable the background noise filter for better audio output clarity.

The product backlog outlines the key user stories that define the core functionalities of the EdgeSign system. These user stories capture the requirements for gesture recognition, text and speech conversion, cloud integration, and user interaction features. The backlog serves as a foundation for sprint planning and ensures that development remains focused on delivering an efficient, accessible, and user-centric communication solution.

The product backlog for the proposed EdgeSign: An IoT-Enabled Edge–Cloud Hybrid System for Real-Time Sign Language Translation was developed using Agile principles and is visually represented in Figure 1.1. It outlines all the key user stories that define the system's intended functionality and implementation.

Each user story is designed from a specific user perspective and includes:

- MoSCoW prioritization to highlight the importance of each feature (Must Have, Should Have, etc.)
- Functional requirements such as gesture recognition, gesture-to-text conversion, speech output (TTS), cloud integration, and custom gesture mapping
- Non-functional requirements covering aspects like real-time response, low latency, system reliability, and efficient edge device performance
- Effort estimation and status tracking to support sprint planning, monitoring, and execution

This structured backlog ensures that development remains focused on building a user-centric, accessible, and efficient assistive communication system. It particularly aims to support individuals with speech and hearing impairments by providing a real-time, intelligent, and portable communication solution.

Figure 1.1 represents the MS planner board that has our functional implementation where all the tasks were completed simultaneously once it was done and reported to the guide.

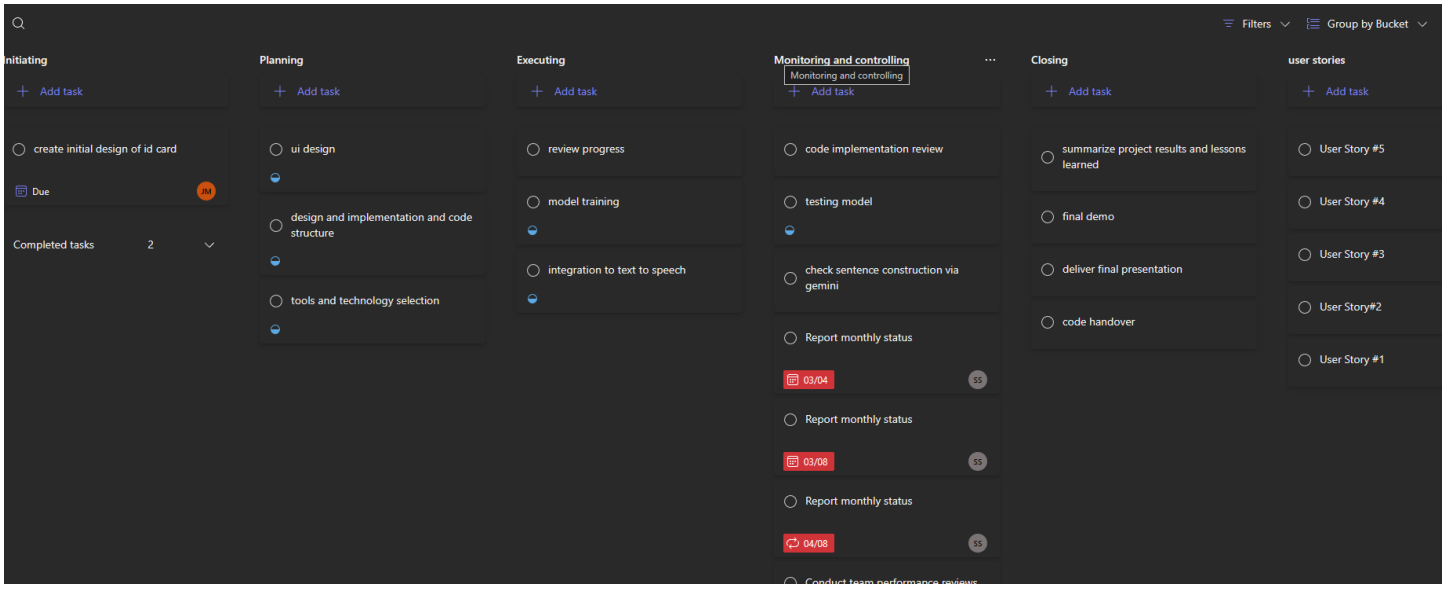


Figure 1.1 MS Planner Board of edge sign translation

1.7 Product Release Plan

The following Figure 1.2 depicts the release plan of the project where the project development was completely planned and executed

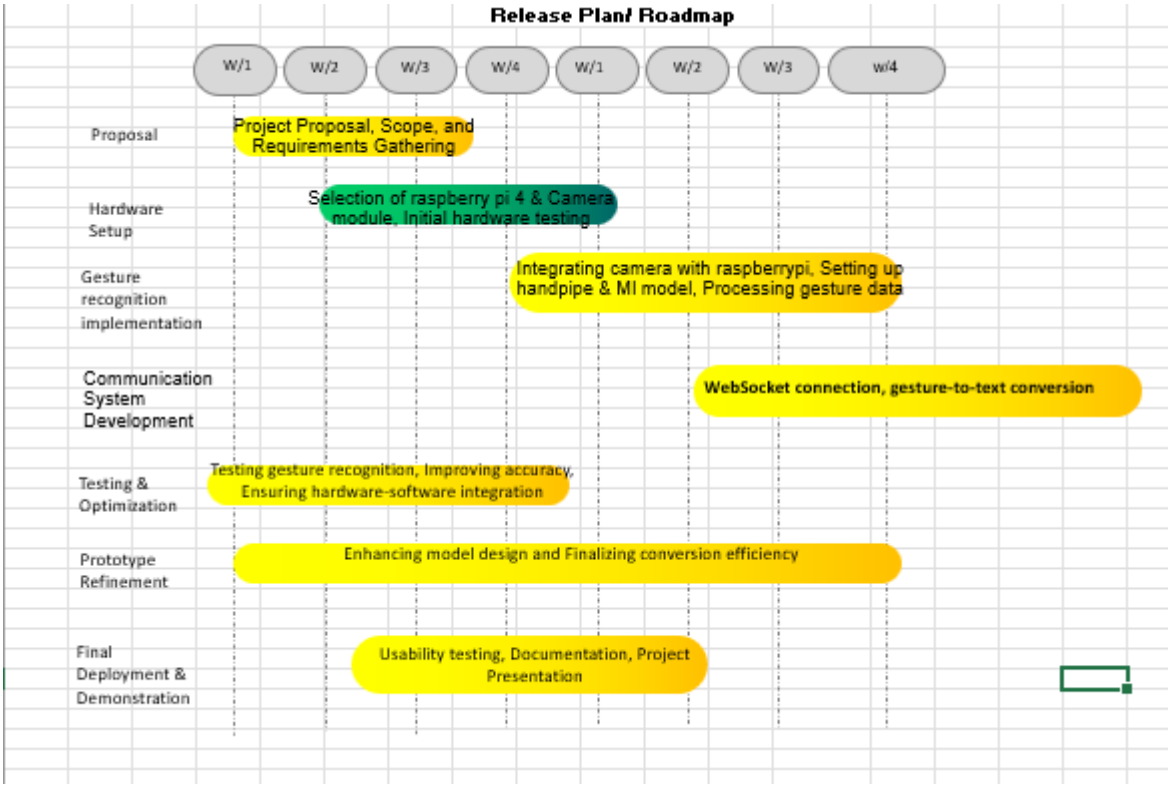


Figure 1.2 Release plan of edge sign language translation

The product release plan illustrated in Figure 1.2 presents a structured roadmap for the development of the EdgeSign system, organized across multiple phases and weekly milestones. The process begins with the proposal phase, where project scope, objectives, and requirements are clearly defined to establish a strong foundation. This is followed by the hardware setup phase, which includes the selection and initial testing of essential components such as the Raspberry Pi 4 and camera module. In the gesture recognition implementation phase, the system is developed to capture hand gestures, extract features using MediaPipe, and process gesture data through a trained machine learning model. The communication system development phase focuses on integrating WebSocket-based communication and implementing gesture-to-text conversion to enable real-time data transmission and interaction.

CHAPTER 2

SPRINT PLANNING AND EXECUTION

2.1 Sprint 1 Gesture Recognition System Setup

2.1.1 Sprint Goal with User Stories of Sprint 1

The goal of Sprint 1 is to set up the project environment and implement the core gesture recognition module on the Raspberry Pi edge device. This includes capturing hand gestures using the Pi Camera, extracting hand landmarks using MediaPipe, and performing initial gesture classification using a machine learning model. The sprint focuses on establishing a functional pipeline for real-time gesture detection and basic communication.

The following table 2.1 represents the detailed user stories of the sprint 1 where gesture recognition system setup done.

Table 2.1 – Detailed User Stories of Sprint 1

S. No	Detailed User Stories
US #1	As a user, I want the system to detect and identify my sign language gestures in real time so that I can communicate instantly and naturally. The system captures hand movements through the Raspberry Pi camera and processes them locally using MediaPipe and a trained Random Forest model to recognize predefined gestures with minimal latency.
US #2	As a developer, I want to set up the Raspberry Pi environment with camera integration so that real-time gesture capture can be performed reliably. This includes configuring the Pi Camera, installing required libraries, and ensuring smooth data acquisition.
US #3	As a developer, I want to extract and process hand landmarks using MediaPipe so that meaningful features can be provided to the gesture recognition model for accurate classification.

Planner Board representation of user stories are mentioned below figures 2.1 which depicts the user story in the MS planner along with its priority and detailed information about it.

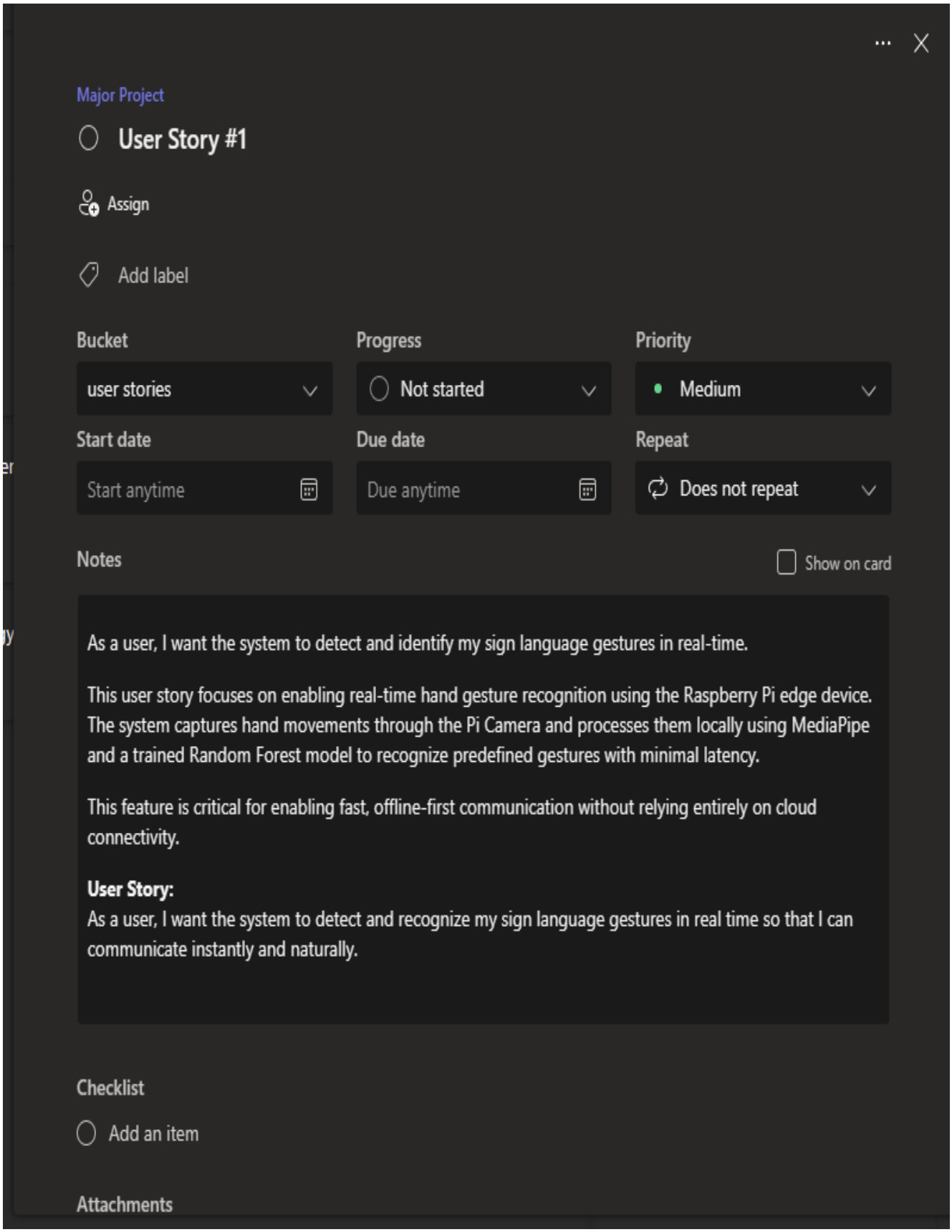


Figure 2.1 User story for Hand gesture detection in camera

2.1.2 Functional Document

2.1.2.1 Introduction

The EdgeSign system is designed to provide an assistive communication solution for individuals with speech and hearing impairments. The system enables real-time translation of hand gestures into text and speech using a combination of computer vision and machine learning techniques. In Sprint 1, the focus is on implementing the core gesture recognition pipeline on the Raspberry Pi edge device. This includes capturing hand gestures using the Pi Camera, extracting hand landmarks using MediaPipe, and classifying gestures using a trained machine learning model.

2.1.2.2 Product Goal

Achieving correct real-time hand gesture recognition and basic text translation for a set of defined gestures is the main objective of this sprint. This underpins a more general assistive communication system, hence satisfying the aim of the project of providing available and inclusive conversational tools. The objective is to create a dependable edge-based processing pipeline running smoothly on the Raspberry Pi with little latency.

Though primarily for persons with speech and hearing impairments, The EdgeSign system also benefits carers and teachers helping them. For communication, these people depend on sign language and need assisting devices to engage more successfully in daily life. Originally designed and tested in confined settings, the system aims to expand its utility to actual uses like schools, healthcare facilities, and public spaces.

Several main mechanism propel the system. Real-time processing via the MediaPipe platform, which extracts hand landmarks, is done with a camera setup. By means of a machine learning model like a Random Forest classifier, which links these markers to established gesture categories, gesture categorization is accomplished. The acknowledged motions are subsequently translated into textual output, which forms the basis for more processing in following steps. This method guarantees correct and quick recognition with little computational load on the edge device.

2.1.2.3 Authorization Matrix

The following table 2.2 represents the Authorization Matrix, User Level of the Product

Table 2.2 Authorization Matrix

ROLE	ACCESS LEVELS
END USER	Uses the system to perform hand gestures and receive text or audio output in real time.
MOBILE PHONE USERS	Views recognized text, manages gesture mappings, and monitors system activity.
ADMIN	Adds or edits gesture datasets, manages model updates, and monitors system performance.

2.1.2.4 Features

Feature 1 – Edge–Cloud Gesture Processing

- Description

  The system captures hand gestures in real time through the Raspberry Pi camera, utilizes the MediaPipe framework to accurately detect and extract hand landmarks, processes gesture data locally on the edge device for low-latency response, and securely transmits the processed information to the cloud using WebSocket communication for advanced contextual analysis, refined sentence generation, improved recognition accuracy, and scalable performance enhancement. This edge–cloud hybrid architecture ensures efficient resource utilization while balancing speed, portability, and intelligent processing capabilities for assistive communication.

- User Story

  As a user, I want my hand gestures to be captured, processed, and transmitted efficiently through edge–cloud communication so that the system can recognize gestures accurately and deliver real-time results.

2.1.2.5 Assumptions

- Raspberry Pi 4, camera module, and required hardware components will be available and functional throughout the sprint.
- The MediaPipe framework will accurately extract hand landmarks under normal lighting conditions.
- The machine learning model (Random Forest) will provide reliable gesture classification for predefined gestures.
- Stable communication between the Raspberry Pi and cloud services will be available for data transmission.
- Team members have sufficient knowledge of Python, machine learning, and embedded systems for development.
- The system will operate with acceptable latency for real-time gesture recognition and output generation.

The system assumes that the Raspberry Pi 4, camera module, and all required hardware components are available and function reliably throughout the sprint. It is expected that the MediaPipe framework can accurately extract hand landmarks under normal lighting conditions and that the Random Forest model will provide consistent gesture classification for predefined inputs. Stable communication between the Raspberry Pi and cloud services is assumed for seamless data transmission. Additionally, the development team is expected to have sufficient expertise in Python, machine learning, and embedded systems. The system is also assumed to operate with acceptable latency to support real-time gesture recognition and output generation.

Several main mechanism propel the system. Real-time processing via the MediaPipe platform, which extracts hand landmarks, is done with a camera setup. By means of a machine learning model like a Random Forest classifier, which links these markers to established gesture categories, gesture categorization is accomplished. The acknowledged motions are subsequently translated into textual output, which forms the basis for more processing in following steps. This method guarantees correct and quick recognition with little computational load on the edge device.

2.1.3 Architecture Document

2.1.3.1 Application

Microservices:

We selected Microservices Architecture for EdgeSign cloud hybrid sign language translation System because of the following reasons:

- Scalability:
  Microservices allow individual services such as gesture recognition, cloud processing, and text-to-speech to be scaled independently based on workload demands, ensuring efficient resource usage in the EdgeSign system.
- Flexibility:
  Each service can be modified or upgraded without impacting other components, enabling easy feature additions such as new gesture models or enhanced cloud intelligence.
- Technology-Diversity:
  Different technologies and frameworks can be used for different services (e.g., Python for edge processing, cloud APIs for AI inference), allowing the system to leverage the most suitable tools for each task.
- Faster Development and Deployment:
  Independent development of services enables parallel work by team members, reducing development time and allowing faster testing and deployment cycles.
- Improved Maintainability:
  Isolated services simplify debugging, testing, and maintenance, making the system easier to manage and update over time.

2.1.3.2 Detailed Diagrams

Use Case Diagram

- User-performed sign language gestures initiate and trigger the communication system for real-time processing.
- The Text-to-Speech (TTS) module generates clear, natural, and understandable audio output for effective real-time communication.
- The gesture recognition module developed in Sprint 1 provides accurate and dependable input for sentence formation and translation
- Cloud services remain available, responsive, and efficient for advanced sentence generation,

Figure 2.2 The use case diagram illustrates the interaction between the user and the EdgeSign system for real-time gesture-based communication. It shows how hand gestures are captured, processed, and converted into text and speech outputs through edge and cloud components. The diagram also highlights system features such as mobile app integration, audio output, and user-controlled settings.

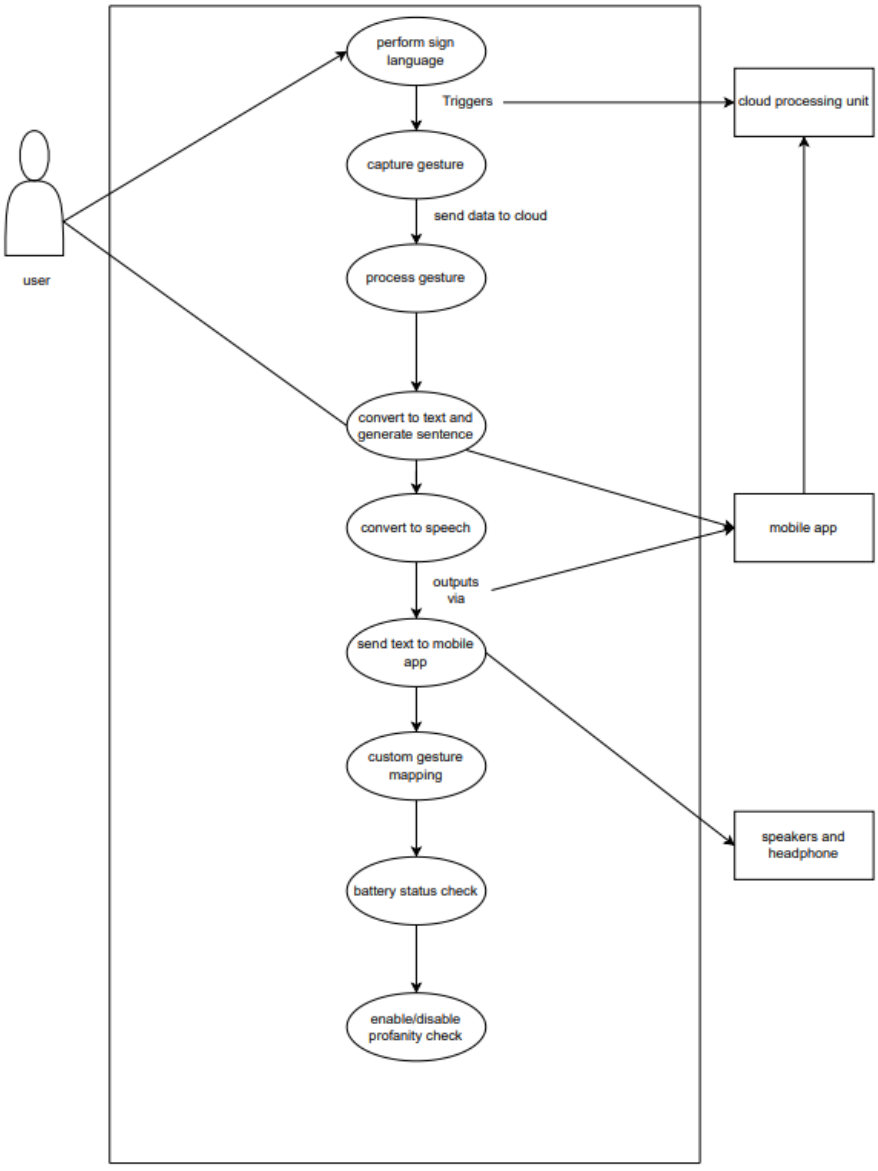


Figure 2.2 Use case diagram of edge sign translation

2.1.4 UI DESIGN

Figure 2.3 and 2.4 illustrates the ui design of the respective home page and hand detection

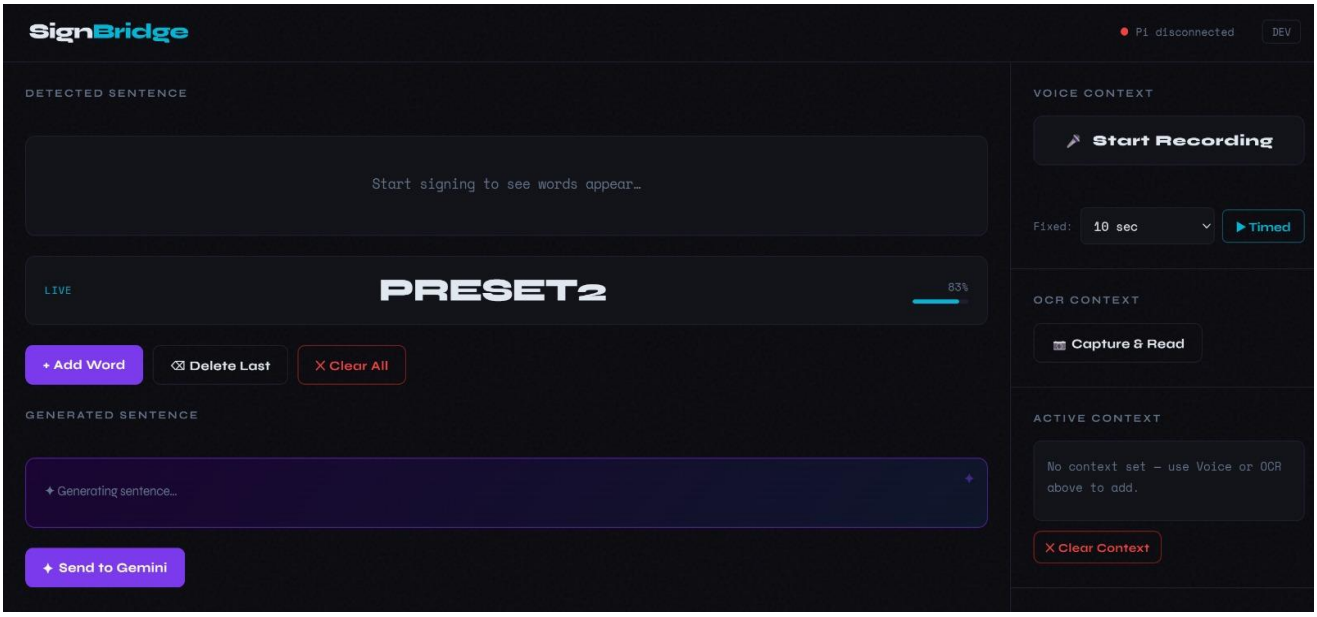


Fig 2.3 UI Design of Home Page

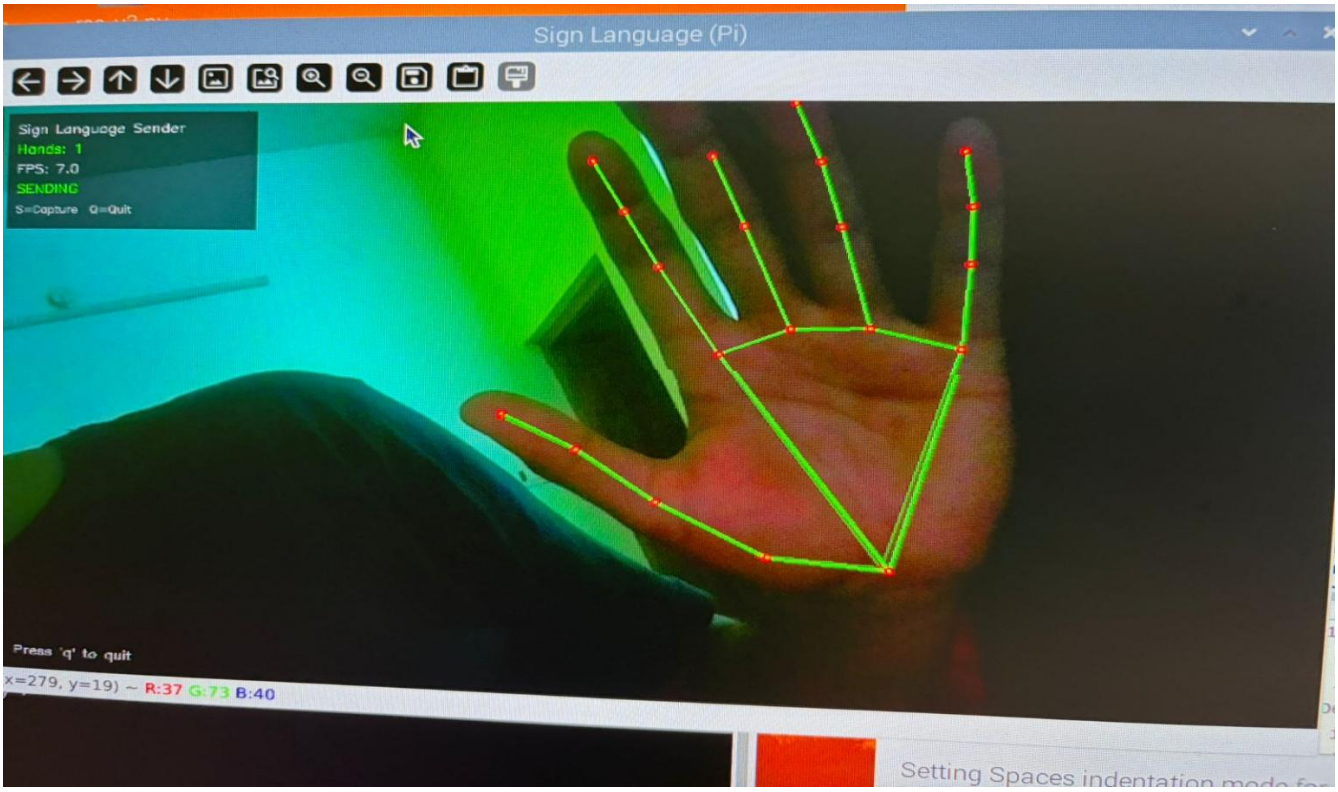


Fig 2.4 Hand Tracking And Landmark Detection

2.1.5 Functional Test Cases

The following table 2.3 represents the detailed functional Test Case of the sprint 1

Table 2.3 Detailed Functional Test Case

		Functional Test Case Template				
Feature	**Test Case**	**Steps to execute test case**	**Expected Output**	**Actual Output**	**Status**	**More Information**
OCR Text Extraction	Photo to Text Recognition	1. Open OCR module in the system. 2. Capture an image containing printed text using the camera. 3. Send image to OCR engine for processing.	Text from the captured image is extracted and displayed correctly.	Text extracted successfully from image.	Pass	Uses OCR engine (e.g., Tesseract) for extracting printed text from images.
Speech Recognition	Convert Speech to text using audio	1. Start the speech recognition module. 2. Speak a sentence through the microphone. 3. Audio is captured using PyAudio and processed by the speech recognition engine.	Spoken words are converted into text and displayed on the interface.	Speech converted into text successfully.	Pass	Uses PyAudio for audio capture and speech recognition model/API for transcription
Developer Mode	Preview Sample & Data log	1. Enable Developer Mode in the system settings. 2. Perform gestures or input data. 3. View preview window and generated data logs.	System displays gesture preview and stores data logs for debugging.	Preview and logs displayed correctly.	Pass	Used for debugging and monitoring gesture detection, model predictions, and system performance.
Gesture Recognition	Recognize Predefined Gesture	1. Start EdgeSign application on Raspberry Pi.2. Show a trained hand gesture in front of the Pi Camera.	Gesture is detected and classified correctly.	Gesture recognized correctly.	On going	Uses MediaPipe hand landmarks and Random Forest classifier.
Gesture Identification	Detect Unknown Gesture	1. Start application.2. Show an untrained or random hand gesture.	No output or low-confidence prediction shown.	No gesture detected.	On going	Unknown gestures are ignored to avoid false positives.

2.1.6 Committed Vs Completed User Stories

The following figure 2.5 represents the committed Vs Completed User Stories of sprint 1

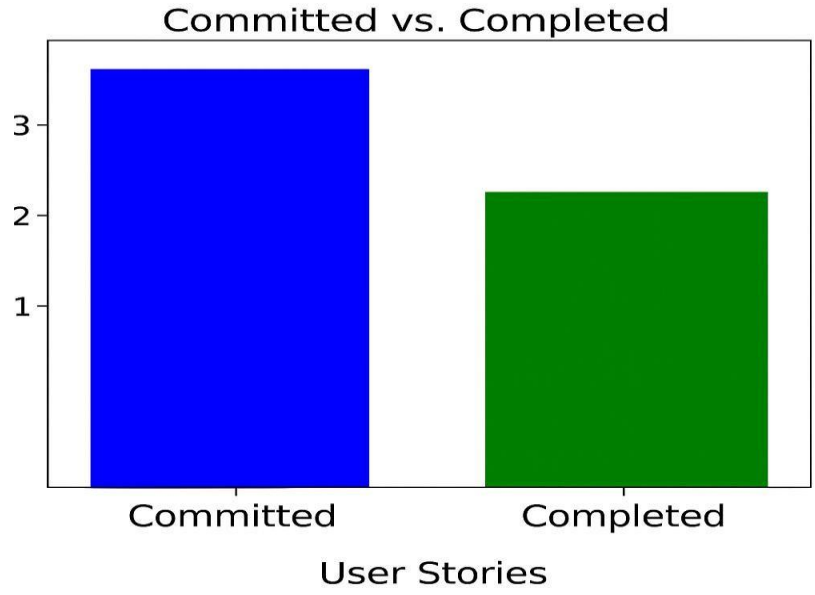


Figure 2.5 Bar graph for Committed Vs Completed User Stories

2.1.7 Sprint Retrospective

Sprint 1 focused on establishing the foundational environment for the EdgeSign system and implementing the core gesture recognition module on the Raspberry Pi. The team began by setting up the hardware components, including the Raspberry Pi 4, Pi Camera, and a portable power setup. The software environment was configured using Python, OpenCV, and MediaPipe. The MediaPipe Hands module was utilized for real-time hand landmark extraction, which enabled accurate detection of hand gestures under normal lighting conditions. A machine learning model (Random Forest) was used to classify predefined gestures, forming the base of the recognition pipeline. Version control using Git was also established to manage development and track progress efficiently.

However, several challenges were encountered during the sprint. Gesture recognition accuracy was affected under low-light conditions, and minor frame processing delays were observed during continuous gesture input. Additionally, the initial dataset required refinement to improve classification consistency. These issues highlighted the importance of proper dataset preparation and environmental considerations. The team also recognized the need for modular testing, separating gesture detection and classification components to simplify debugging and improve system reliability.

Despite these challenges, Sprint 1 successfully delivered a working prototype capable of real-time gesture recognition. The sprint provided valuable insights into system limitations, particularly regarding lighting sensitivity and dataset diversity. It also emphasized the need for optimizing performance on edge devices.

Moving forward, the next sprint will focus on integrating cloud communication, implementing gesture-to-text sentence generation, and adding text-to-speech (TTS) functionality. Additional improvements will include expanding the dataset, optimizing model performance, and enabling interaction with a mobile application. Overall, Sprint 1 laid a strong foundation for building a scalable and efficient assistive communication system.

2.2 SPRINT 2 Communication & Output Integration

2.2.1 Sprint Goal with User Stories of Sprint 2

The goal of Sprint 2 is to extend the system by enabling gesture-to-sentence formation, integrating text-to-speech (TTS) for audio output, and establishing communication with the mobile application. This sprint focuses on improving user interaction by providing both visual and audio outputs, along with cloud-based processing for enhanced functionality.

The following table 2.4 represents the detailed user stories of the sprint 2

Table 2.4 – Detailed User Stories of Sprint 2

S. No	Detailed User Stories
US #4	As a user, I want the device to play the translated text through a speaker so that I can communicate verbally with others. The system converts recognized text into speech using a TTS module and outputs it through speakers or Bluetooth audio devices.
US #5	As a user, I want the recognized word or sentence to be displayed on the mobile UI so that I can visually confirm and track my communication. The system sends text data from the Raspberry Pi to the mobile application using WebSocket or BLE communication.
US #6	As a user, I want the system to generate meaningful sentences from recognized gestures so that communication becomes more natural and context-aware.

Planner Board representation of user stories are mentioned below figure 2.6 where the respective user story is shared in the MS planner along with its priority and detailed information is shared.

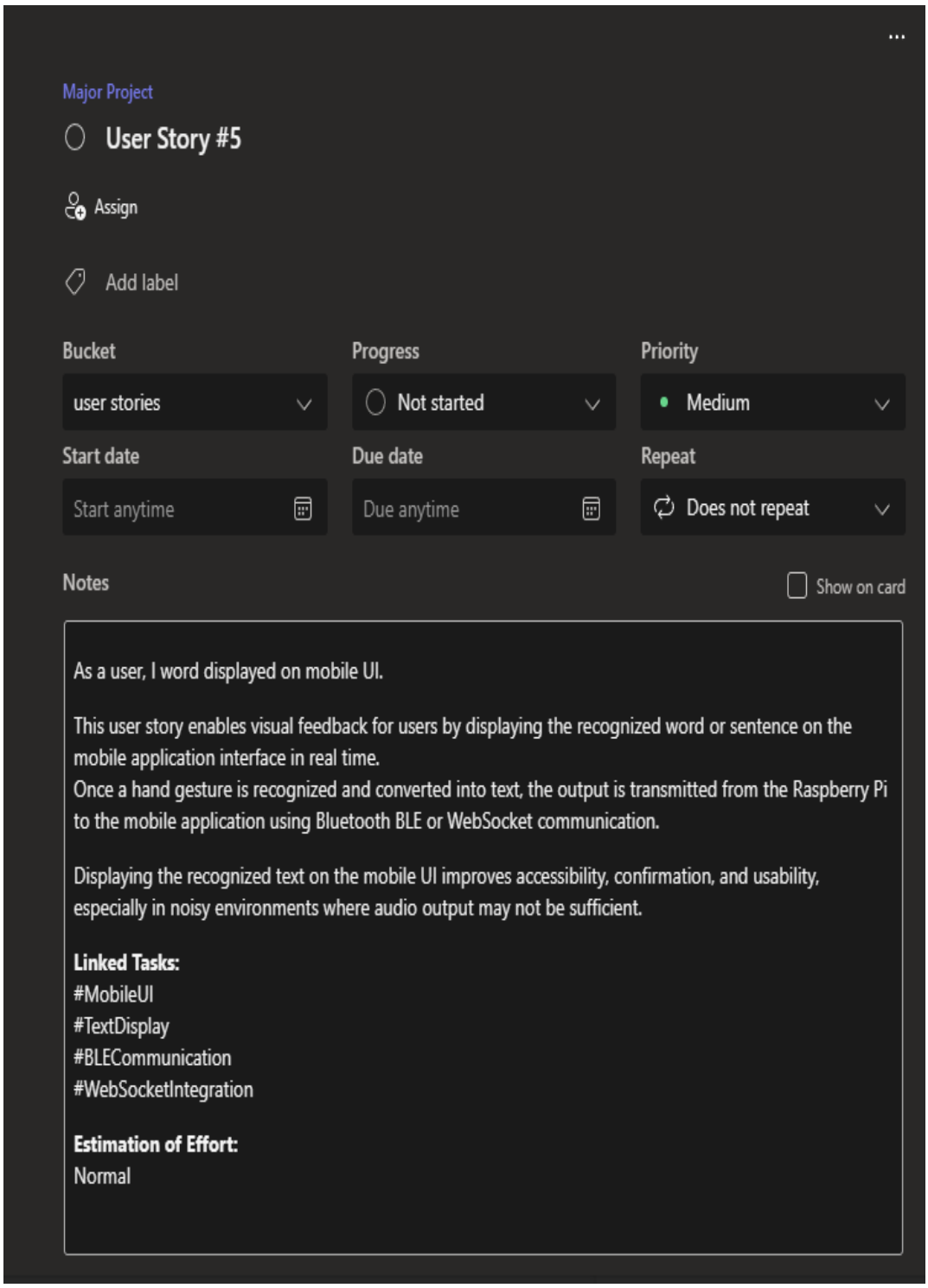


Figure 2.6 user story for sentence conversion in camera

2.2.2 Functional Document

2.2.2.1 Introduction

Gesture-to-Speech and UI Integration Description

Once a gesture is recognized and classified into a word or phrase, the system converts it into speech using a Text-to-Speech (TTS) engine. The generated audio is played through speakers or wireless audio devices such as Bluetooth headphones. At the same time, the recognized text is transmitted from the Raspberry Pi to the mobile application using WebSocket or BLE communication, where it is displayed in real time.

2.1.2.2 Features

Feature 1 – AI-Based Sentence Generation

- Description:

Recognized gestures are converted into words and combined into meaningful sentences. Cloud based AI processing enhances sentence structure and context.

- User Story:

As a user, I want the system to convert gestures into meaningful sentences that becomes natural and understandable.

2.2.2.3 Assumptions

- Stable WebSocket or BLE communication ensures reliable and seamless connectivity between the Raspberry Pi edge device and the mobile application.
- The Text-to-Speech (TTS) module generates clear, natural, and understandable audio output for effective real-time communication.
- The gesture recognition module developed in Sprint 1 provides accurate and dependable input for sentence formation and translation.
- Cloud services remain available, responsive, and efficient for advanced sentence generation, contextual processing, and data handling.
- The mobile application is fully compatible and capable of displaying real-time text updates for smooth user interaction and monitoring

2.2.3 Architecture Document

- Raspberry Pi 4 acts as the central edge device, capturing gestures and coordinating communication between components.
- Gesture Recognition Module (MediaPipe + ML Model) processes hand gestures by extracting hand landmarks using MediaPipe and classifying them using a trained Random Forest model. The Pi Camera captures real-time hand movements.
- Cloud Processing Unit receives gesture data from the Raspberry Pi via WebSocket communication and performs advanced processing such as sentence generation and contextual enhancement.
- Audio Output Module (Speaker / Bluetooth Audio) converts the generated text into speech using a Text-to-Speech (TTS) engine and plays it through speakers or Bluetooth-enabled devices.
- Communication Module (WebSocket / BLE) enables data transfer between the Raspberry Pi and the mobile application for real-time text display and system interaction.
- Mobile Application displays recognized text, allows custom gesture mapping, and provides controls such as enabling or disabling features like profanity filtering.
- Data Storage Module stores gesture datasets, mappings, and logs for improving system performance and training.
- Power/Battery Module supplies power to the Raspberry Pi and monitors battery levels to ensure uninterrupted operation.

The architecture diagram illustrates the complete EdgeSign system workflow, where the Raspberry Pi 4 serves as the core edge device responsible for real-time gesture capture, hand landmark detection using MediaPipe, and local machine learning-based gesture recognition. By adopting an edge–cloud hybrid architecture, the system processes critical gesture inputs locally for low-latency performance while transmitting gesture data to the cloud through WebSocket communication for advanced contextual sentence generation, enhanced language processing, and improved accuracy. The cloud layer strengthens the system by refining gesture outputs into meaningful sentences and enabling scalable AI-powered enhancements. Finally, the processed output is delivered to users through a mobile application as real-time text updates and through audio devices using Text-to-Speech technology, ensuring seamless, accessible, and efficient communication for individuals with speech or hearing impairments across everyday environments.

Figure 2.7 depicts the system architecture integrates gesture recognition, real-time processing, local storage, and wireless audio output on a Raspberry Pi 4 to enable seamless communication for speech- and hearing-impaired users.

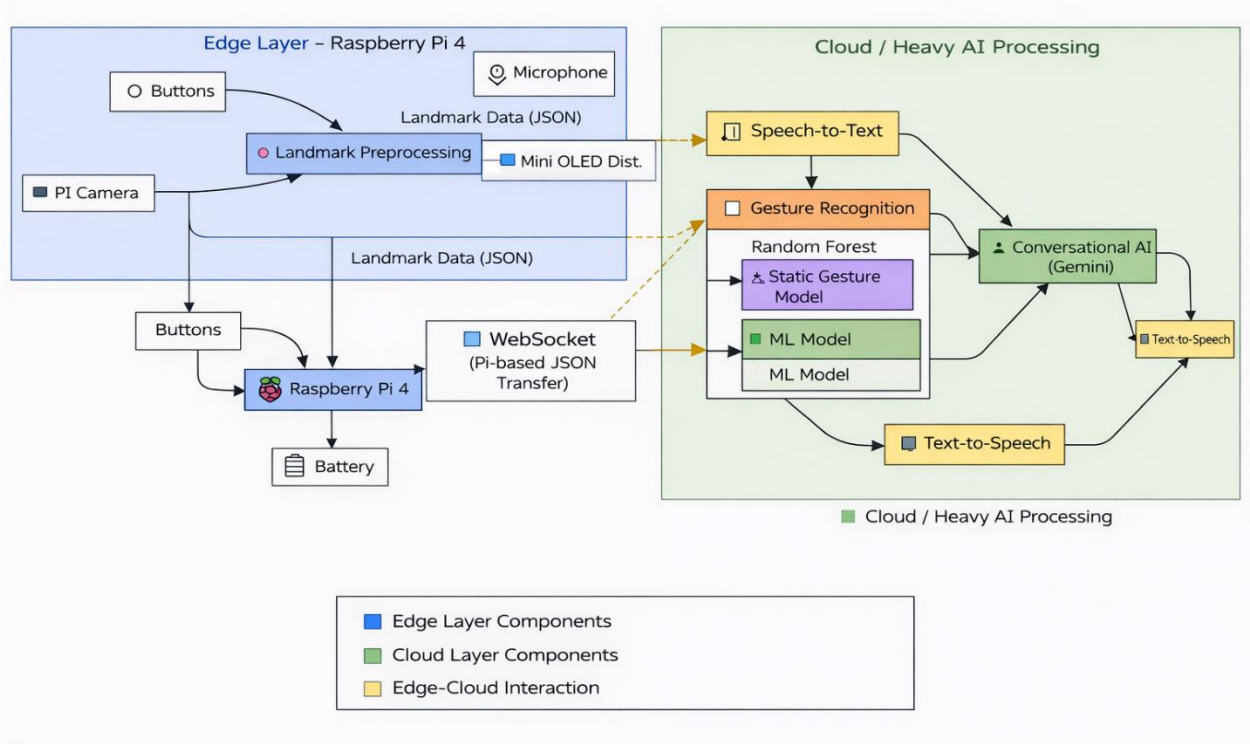


Figure 2.7 Architecture diagram of the edge cloud sign translation system

The system architecture follows a modular design where the Raspberry Pi 4 acts as the central processing unit, handling gesture capture, preprocessing, and initial classification at the edge. The processed data is communicated to the cloud through WebSocket for advanced processing such as sentence generation and contextual enhancement. The architecture also integrates output modules, including web app UI and text-to-speech audio output, ensuring efficient, real-time, and user-friendly communication.

2.2.4 UI Design

Figure 2.8 shows the Dev panel for remote debugging where the display board is connected to the UI which shows the frame rate and ocr results along with the audio input details.

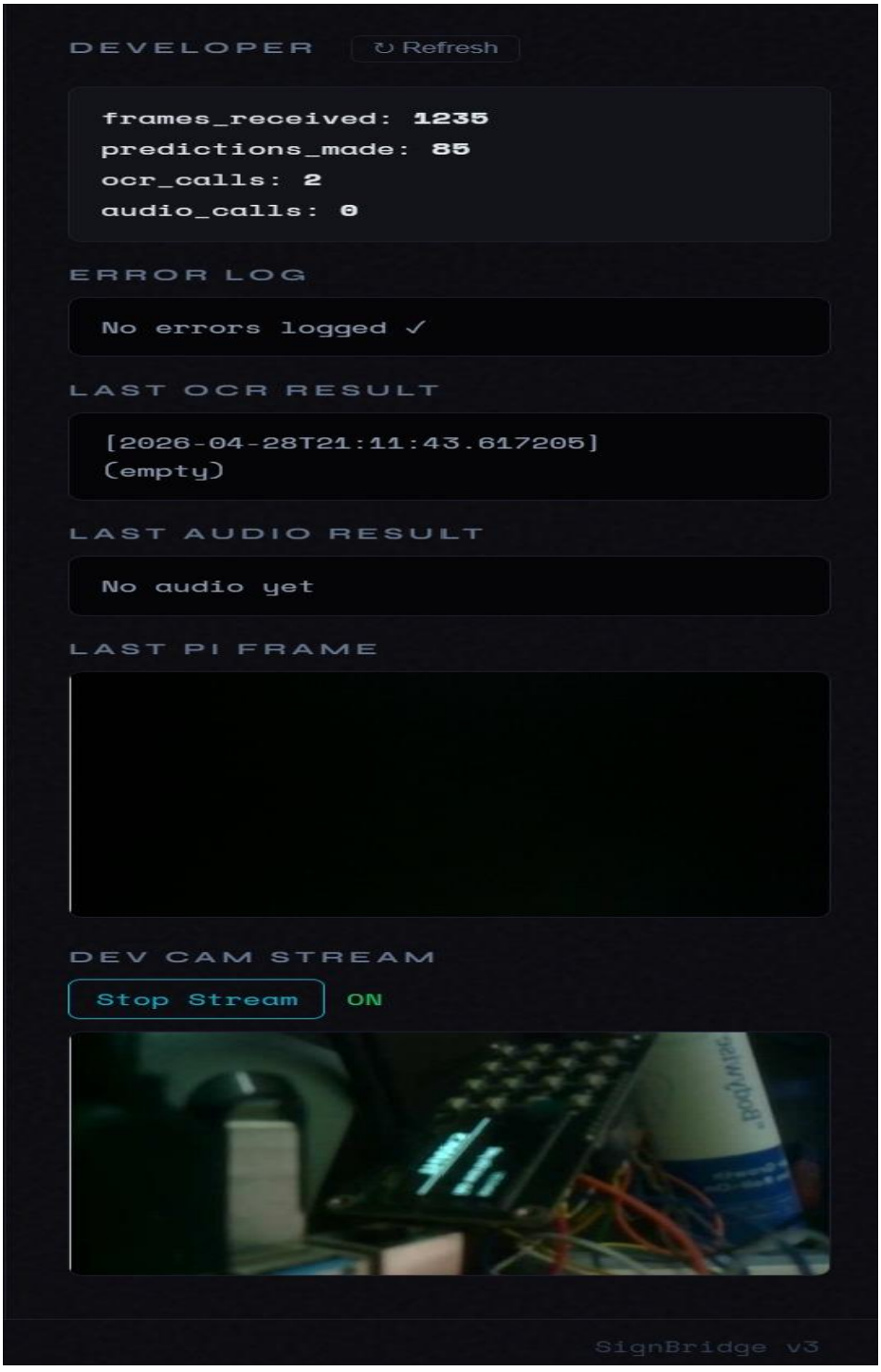


Fig 2.8 Dev panel for remote debugging

2.2.5 Functional Test Cases

Table 2.5 Functional Test Case of Sprint 2

Feature	Test Case	Steps to execute test case	Expected Output	Actual Output	Status	More Information
OCR Text Extraction	Photo to Text Recognition	1. Open OCR module in the system. 2. Capture an image containing printed text using the camera. 3. Send image to OCR engine for processing.	Text from the captured image is extracted and displayed correctly.	Text extracted successfully from image.	Pass	Uses OCR engine (e.g., Tesseract) for extracting printed text from images.
		1. Start the speech recognition module.				
Speech Recognition	Convert Speech to text using audio	2. Speak a sentence through the microphone.	Spoken words are converted into text and displayed on the interface.	Speech converted into text successfully.	Pass	Uses PyAudio for audio capture and speech recognition model/API for transcription
		3. Audio is captured using PyAudio and processed by the speech recognition engine.				
Developer Mode	Preview Sample & Data log	1. Enable Developer Mode in the system settings.	System displays gesture preview and stores data logs for debugging.	Preview and logs displayed correctly.	Pass	Used for debugging and monitoring gesture detection, model predictions and system performance
		2. Perform gestures or input data.				

2.2.6 Committed vs Completed User Stories

In Figure 2.9 The chart compares the number of user stories committed versus those successfully completed during the sprint execution

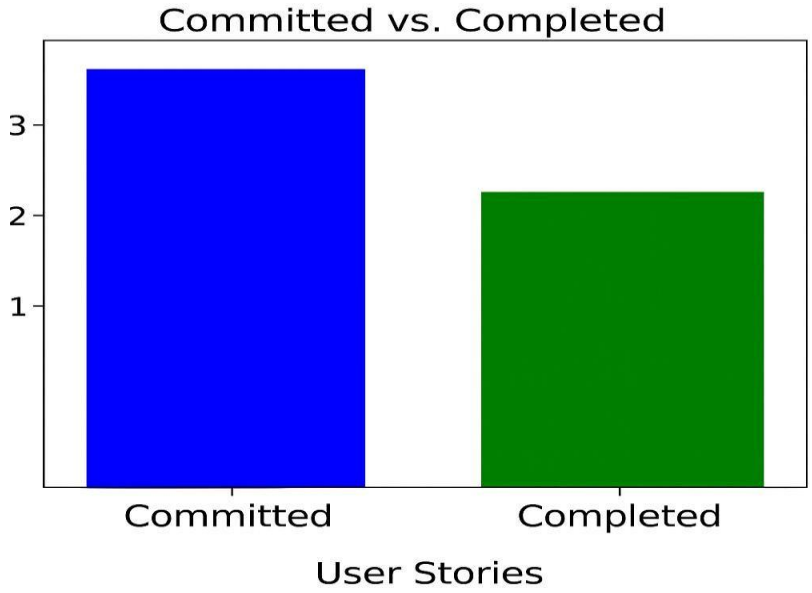


Figure 2.9 Bar graph for Committed Vs Completed User Stories

2.2.7 Sprint Retrospective

Sprint 2 focused on enhancing the prototype from a basic gesture recognition system to a more complete communication solution capable of generating sentences and delivering both visual and audio outputs. The primary goal was to design a system that could process sequences of recognized gestures and convert them into meaningful text, while also enabling real-time interaction through a mobile interface. The integration of WebSocket-based communication allowed seamless transmission of recognized text from the Raspberry Pi to the mobile application, where users could view and track the generated output.

In parallel, the team worked on integrating Text-to-Speech (TTS) functionality to convert the generated text into audio output. Once a gesture or sequence of gestures was recognized and processed, the system generated speech output, which was played through speakers or Bluetooth-enabled devices. This significantly improved the usability of the system by enabling users to communicate verbally without relying on external assistance.

2.3 SPRINT 3 Advanced Feature Integration & System Optimization

2.3.1 Sprint Goal with User Stories of Sprint 2

The goal of Sprint 3 is to enhance the EdgeSign system by integrating advanced communication features such as OCR-based text extraction, Speech-to-Text conversion, and developer monitoring tools. This sprint focuses on improving system usability, debugging capabilities, and overall performance optimization for real-world deployment.

The following table 2.6 represents the detailed user stories of the sprint 3

Table 2.6 – Detailed User Stories of Sprint 3

S. No	Detailed User Stories
US #7	As a user, I want the system to extract printed text from images using OCR so that I can understand written content easily
US #8	As a user, I want to convert spoken words into text using Speech-to-Text so that I can communicate through voice input.
US #9	As a developer, I want a preview and data logging mode so that I can monitor gesture detection and system performance effectively.

Planner Board representation of user stories are mentioned below figure 2.6 where the respective user story is shared in the MS planner along with its priority and detailed information is shared.

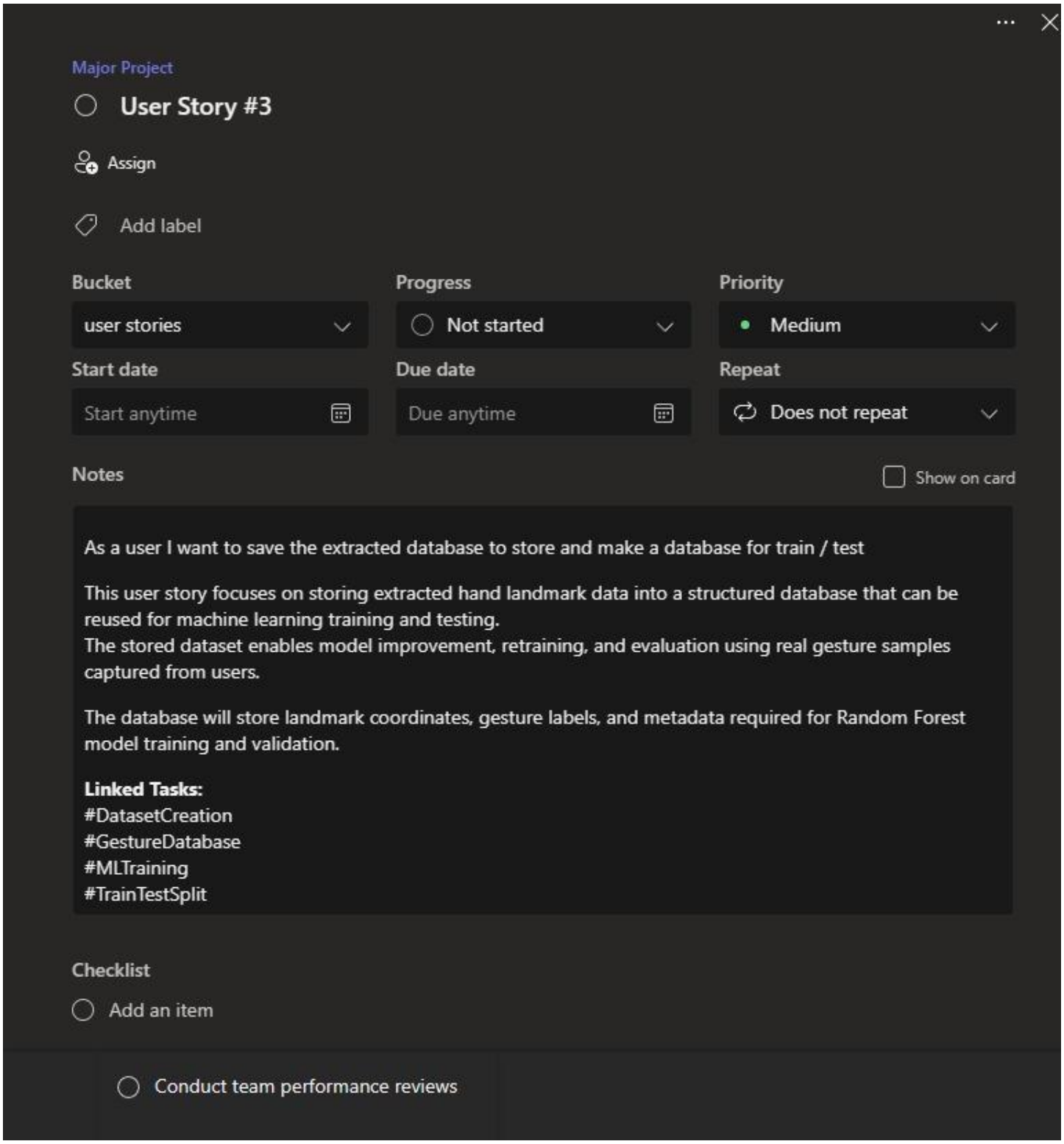


Figure 2.6 user story for sentence conversion in camera

2.3.2 Functional Document

2.2.2.1 Introduction

OCR Text Extraction Description

The OCR module captures images through the camera and extracts printed text using an OCR engine such as Tesseract. The recognized text is displayed on the interface for user interaction and accessibility.

Speech-to-Text-Description

The Speech-to-Text module converts spoken audio into text using a microphone and speech recognition engine. This enables voice-based interaction and improves communication flexibility.

Developer-Mode-Description

The developer mode provides live preview, confidence scores, and system logs for monitoring gesture recognition and debugging. This feature helps improve system performance and testing efficiency.

2.1.2.2 Features

Feature 1 – OCR Text Extraction

Description:

The system captures printed or written text from images using the camera and processes it through OCR technology to extract readable text. The extracted text is then displayed on the interface for user accessibility and interaction.

User Story:

As a user, I want the system to read printed text from images so that I can access written information easily.

Feature 2 – Speech-to-Text

Description:

The system captures spoken audio through a microphone and converts speech into text using a speech recognition engine. This enables users to communicate using voice input and provides flexible real-time text conversion.

User Story:

As a user, I want spoken words to be converted into text so that I can communicate using voice input.

Feature 3 – Developer Mode

Description:

The developer mode provides advanced monitoring tools such as live camera preview, gesture confidence scores, and real-time system logs. This feature supports debugging, testing, and performance optimization for developers.

User Story:

As a developer, I want to monitor gesture predictions and logs in real time so that debugging and performance analysis become easier.

2.2.2.3 Assumptions

- Stable WebSocket or BLE communication ensures reliable and seamless connec

The camera module captures clear images and video frames to ensure accurate OCR text extraction and gesture recognition performance.

OCR engines such as Tesseract provide reliable text recognition for printed and readable visual content.

The speech recognition engine accurately converts spoken words into text under normal audio conditions.

Developer mode logging and monitoring tools function continuously without affecting system performance or real-time responsiveness.

The user interface supports seamless display of OCR results, speech-to-text outputs, and developer diagnostics for effective accessibility and system management.

2.3.3 Architecture Document

Sequence Diagram of the EdgeSign system, demonstrating the interaction flow between the User, Edge Device (Raspberry Pi), Cloud Server, and Mobile Application in delivering real-time gesture-based communication. The process begins when the user performs a hand gesture in front of the camera, which is captured by the Raspberry Pi edge device. The Raspberry Pi then performs initial preprocessing and hand landmark extraction using MediaPipe to prepare the gesture data for recognition. Once preprocessing is completed, the landmark data is transmitted to the cloud server through WebSocket communication, enabling efficient edge–cloud interaction for advanced processing.

At the cloud layer, machine learning models process the received gesture landmarks along with contextual information to improve recognition accuracy and sentence generation. After processing, the cloud server sends the recognized text output back to the edge device and mobile application. The Raspberry Pi then converts the recognized text into speech using a Text-to-Speech (TTS) engine, allowing the generated audio to be played through connected speakers or Bluetooth headsets. This ensures that user gestures are transformed into both readable and audible communication outputs in real time.

The sequence diagram also highlights the mobile application's role in system customization and user interaction. Users can open or configure the mobile application to manage device settings such as custom gesture mapping, gesture updates, and profanity filtering. Configuration commands are transmitted to the Raspberry Pi via Bluetooth Low Energy (BLE), where the edge device applies the updates and confirms successful implementation. The updated customization results are then displayed in the mobile application, ensuring user-friendly control and personalization. Overall, the sequence diagram emphasizes the modular and interactive design of EdgeSign, integrating edge processing, cloud intelligence, audio output, and mobile customization to create an efficient, scalable, and accessible communication system for speech- and hearing-impaired users.

The sequence diagram illustrates the complete operational workflow of the EdgeSign system, where the user interacts with the Raspberry Pi 4 through gesture input, initiating a sequence of edge processing, cloud intelligence, and output generation. The Raspberry Pi functions as the primary edge device, capturing gestures, performing preprocessing with MediaPipe, and transmitting landmark data to the cloud server through WebSocket communication. The cloud server enhances the system by applying machine learning models for gesture recognition, contextual processing, and sentence refinement, ensuring higher accuracy and meaningful output generation.

Figure 2.7 illustrates the Sequence Diagram of the EdgeSign system, demonstrating the interaction flow between the User, Edge Device (Raspberry Pi), Cloud Server, and Mobile Application in delivering real-time gesture-based communication.

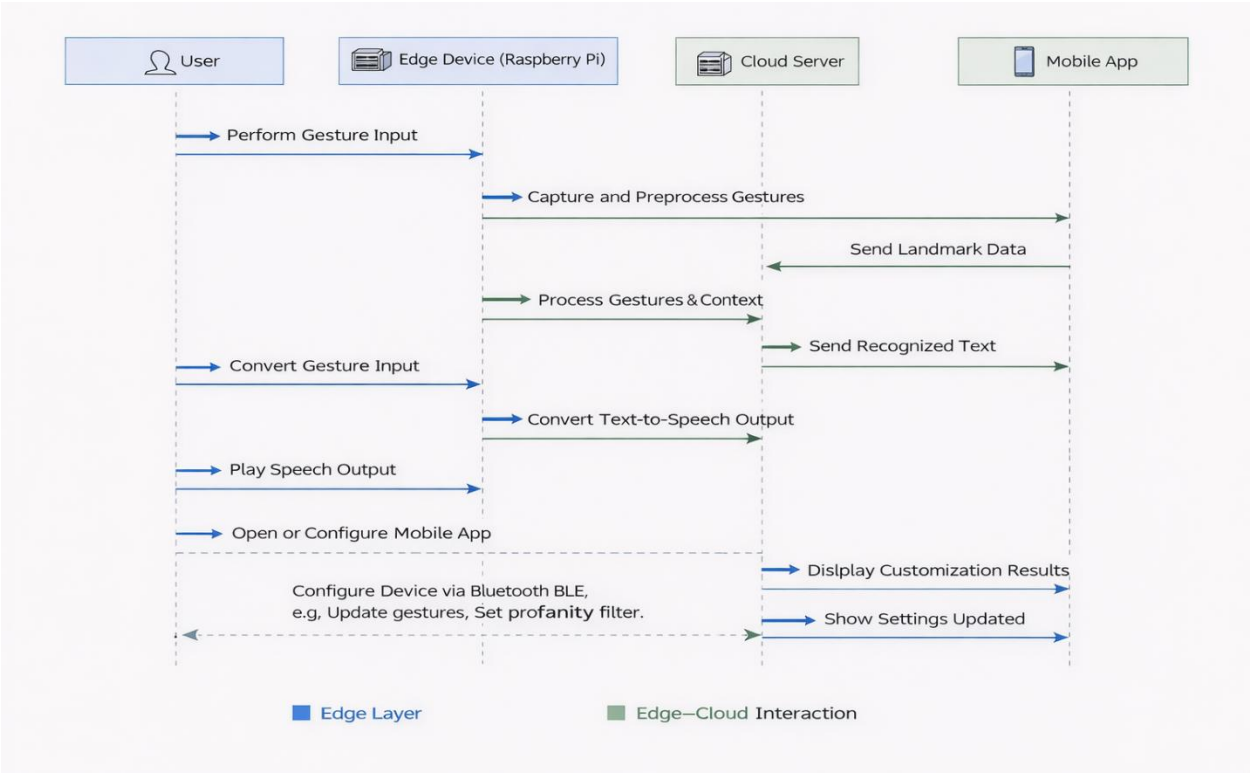


Figure 2.7 Sequence Diagram – Edge System

Once processed, recognized text is returned to the edge layer and mobile application, where it is displayed as real-time text and converted into speech through the Text-to-Speech engine for audio playback. Additionally, the mobile application enables users to customize system settings such as gesture mapping and profanity filtering through BLE communication, allowing adaptive and user-specific system behavior. Overall, the sequence diagram demonstrates the EdgeSign system's modular edge–cloud architecture, integrating gesture recognition, contextual AI processing, real-time text and speech output, and mobile-based customization to deliver an efficient, scalable, and accessible communication platform for speech- and hearing-impaired users.

2.3.4 UI Design

Figure 2.8 shows the OLED interface displaying system outputs, including recognized text, module status, and real-time interaction feedback. It provides users with a compact visual display for monitoring OCR results, speech-to-text conversion, gesture recognition status, and system notifications, ensuring accessible and efficient user interaction.

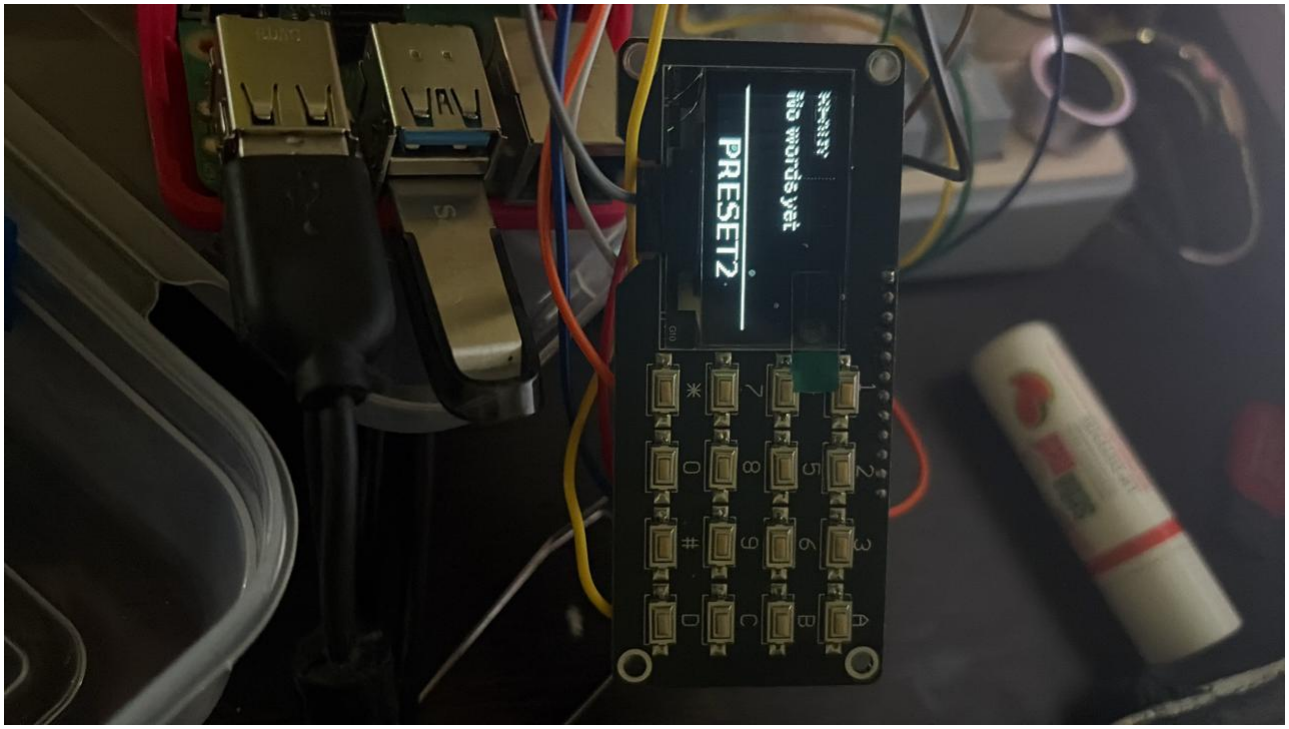


Fig 2.8 OLED -Panel for User-Interface

2.3.5 Functional Test Cases

Table 2.5 Functional Test Case of Sprint 2

Feature	Test Case	Steps to execute test case	Expected Output	Actual Output	Status	More Information
Interface Output Display	Generated outputs are displayed correctly on the interface	1. Execute OCR, Speech-to-Text, or Developer Mode functions. 2. Verify displayed text, logs, or previews on the interface. 3.Button are working	Generated outputs are displayed accurately and in real time on the user interface.	Outputs displayed successfully on interface.	Pass	Validates user accessibility and interface responsiveness for all functional modules.
Communication Stability	System maintains stable communication between modules during testing	1. Perform multiple module interactions simultaneously. 2. Observe data transfer and synchronization.	Stable communication is maintained between system modules without interruption.	Communication remained stable during testing.	Pass	Ensures reliable module interaction and consistent data flow across components.
Developer Mode	Preview Sample & Data log	1. Enable Developer Mode in the system settings. 2. Perform gestures or input data.	System displays gesture preview and stores data logs for debugging.	Preview and logs displayed correctly.	Pass	Used for debugging and monitoring gesture detection, model predictions and system performance

2.3.6 Committed vs Completed User Stories

In Figure 2.9 The chart compares the number of user stories committed versus those successfully completed during the sprint execution

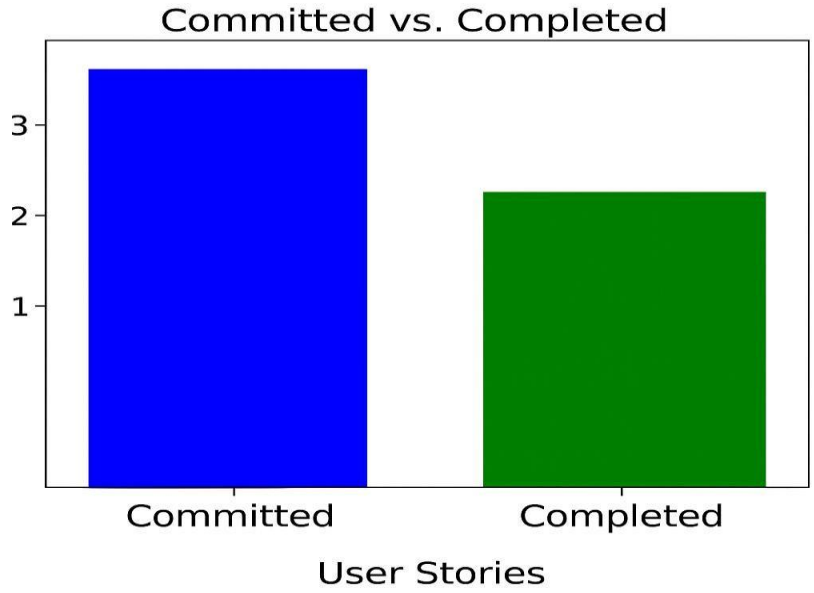


Figure 2.9 Bar graph for Committed Vs Completed User Stories

2.3.7 Sprint Retrospective

Sprint 3 focused on improving the overall usability and functionality of the EdgeSign system through the integration of OCR, Speech-to-Text, and developer monitoring features. The OCR module successfully extracted text from images, while the Speech-to-Text module enabled voice-based communication input. Developer mode improved debugging by providing preview logs and system monitoring tools. Challenges such as speech recognition accuracy in noisy environments and OCR performance under poor lighting conditions were identified during testing. Overall, Sprint 3 enhanced system flexibility, improved testing capabilities, and prepared the project for final deployment and demonstration.

CHAPTER 3

RESULTS AND DISCUSSION

3.1 Project Outcomes

Figure 3.1 The figure represents the user interface of the EdgeSign system, designed for real-time gesture-based communication. It displays detected words, generated sentences, and allows users to manage inputs through simple controls. The interface integrates features such as voice input, OCR capture, and AI-based sentence generation. Overall, it provides an interactive and user-friendly platform for seamless communication.

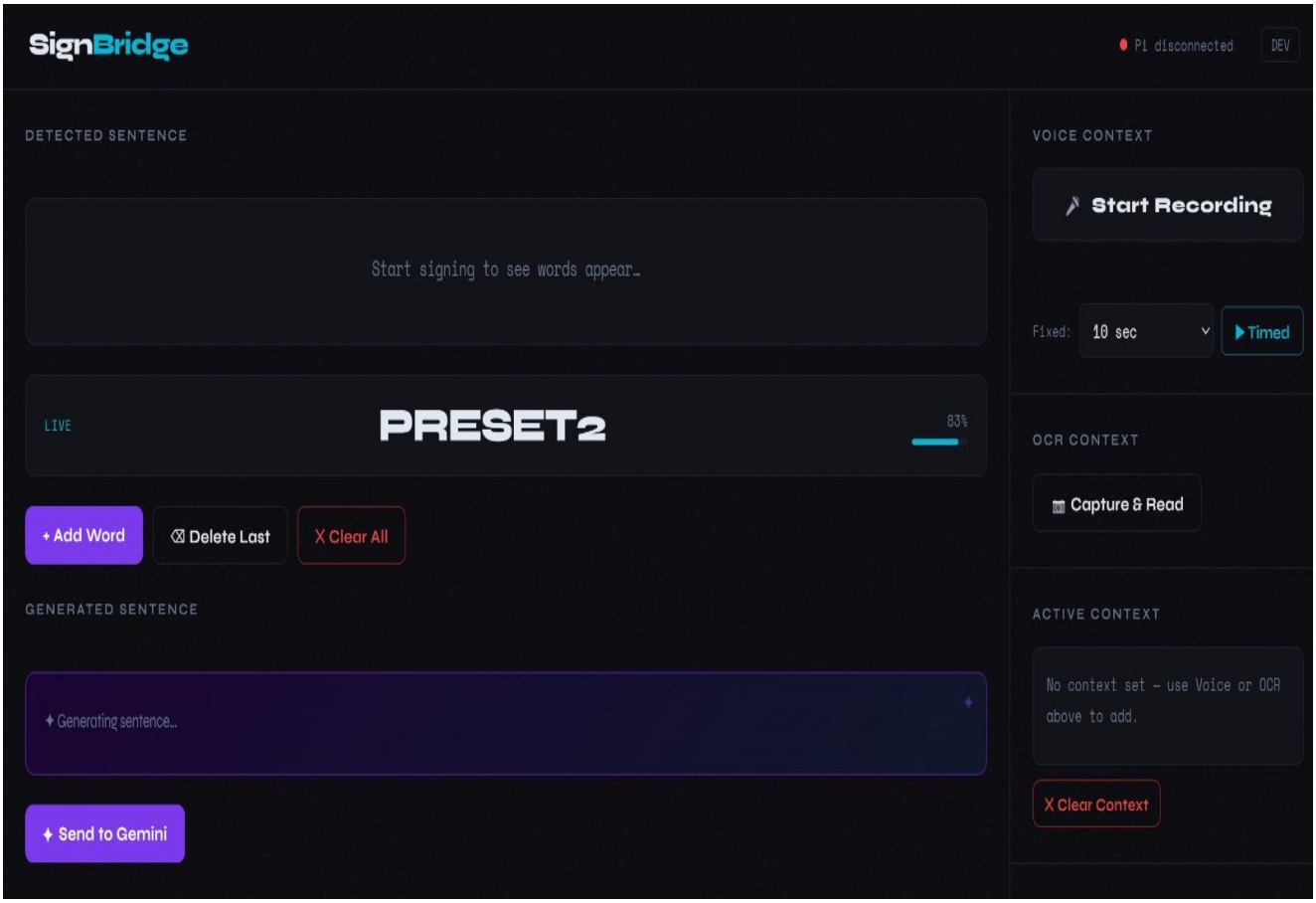


Figure 3.1 UI Front end

Figure 3.2 shows the display buttons of the led display along with in the UI web application

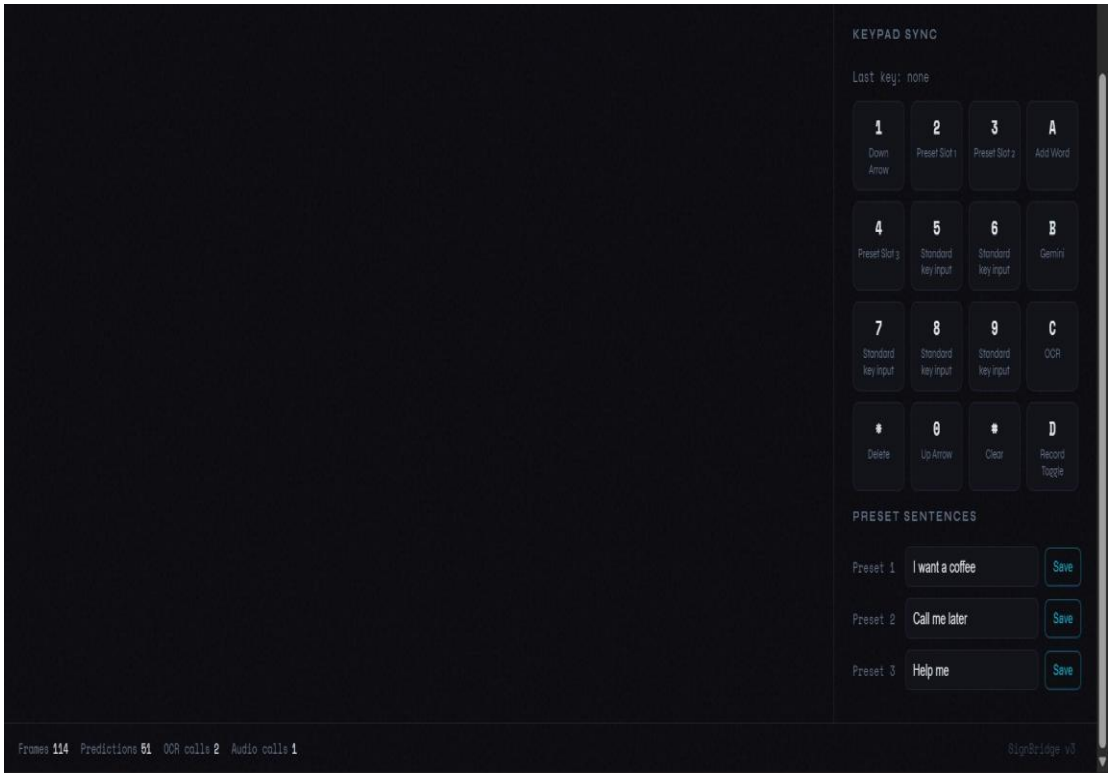


Figure 3.2 led display Integration

Figure 3.3 shows the hand recognition in the camera module where it tracks the finger movements and input landmarks are taken.

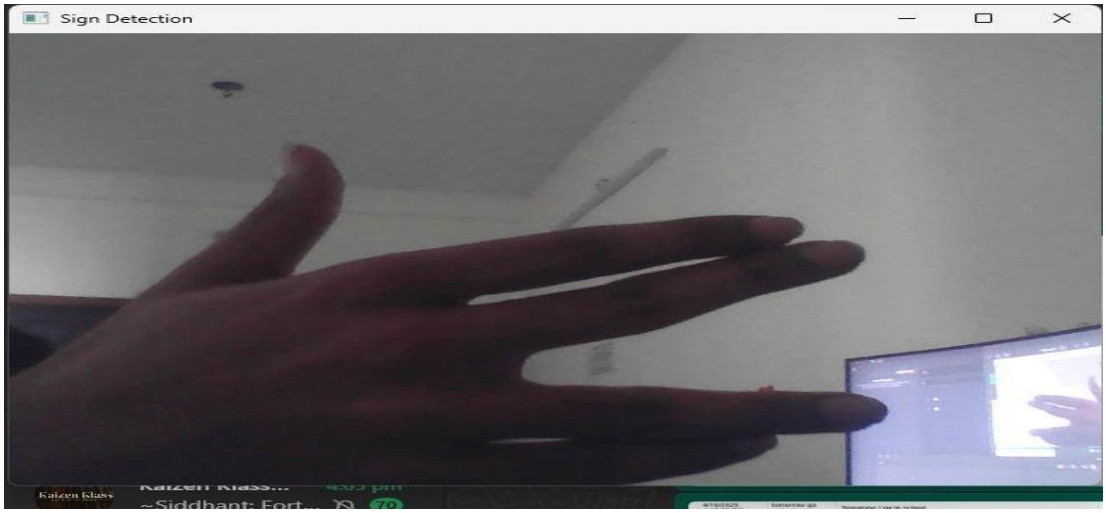


Figure 3.3 Hand gesture Recognition

3.2 Committed Vs Completed User stories

Figure 3.4 contains the bar graph of committed vs completed user stories of edge sign translation

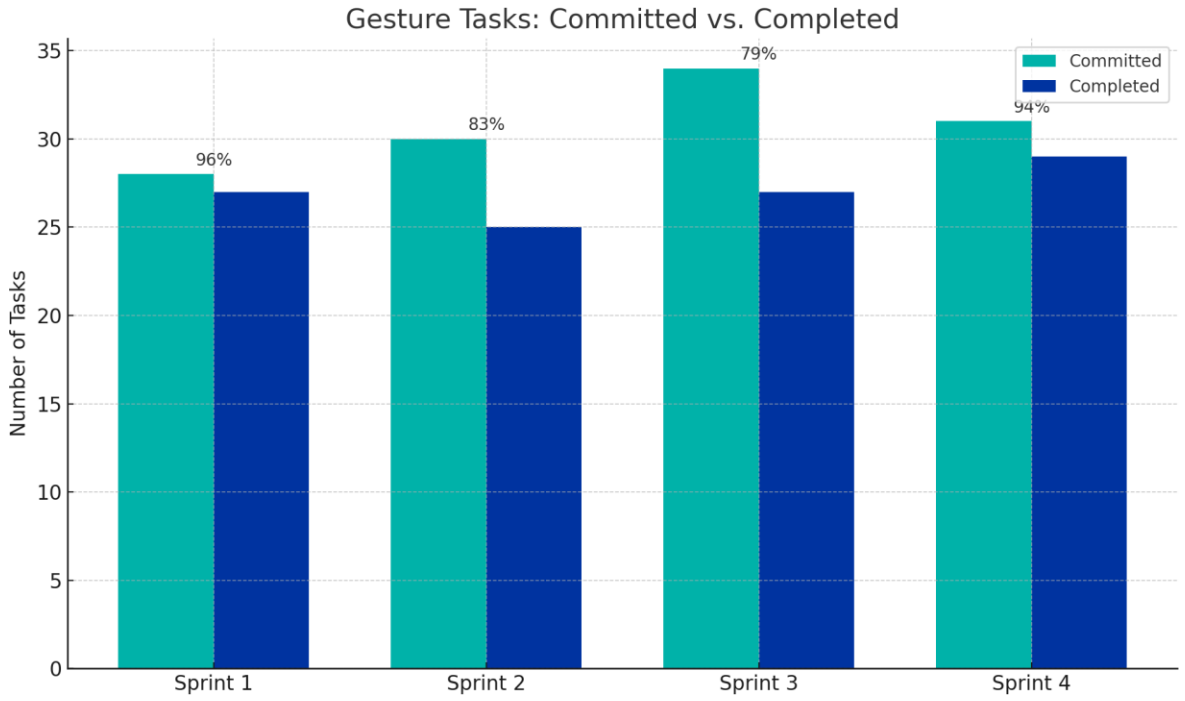


Figure 3.4 Committed vs Completed User Stories

The bar graph illustrates the comparison between committed and completed user stories across different sprints. It shows that Sprint 1 achieved a high completion rate, indicating effective initial planning and execution. In Sprint 2 and Sprint 3, a slight gap between committed and completed tasks can be observed due to increased system complexity and integration efforts. However, the completion rate remains consistently high across all sprints. Sprint 4 demonstrates improved task completion, reflecting better optimization and stability in the system. Overall, the graph highlights steady progress and effective sprint management throughout the project lifecycle.

The graph also indicates improved team coordination and task prioritization in later sprints. The slight variations between committed and completed tasks highlight real-world development challenges. Continuous improvements throughout the sprints contributed to better efficiency and delivery outcomes. Overall, the results demonstrate a well-managed agile process with consistent performance.

CHAPTER 4

CONCLUSION & FUTURE ENHANCEMENTS

Conclusion

One major move toward better communication accessibility for people with speech and hearing problems is the creation of the EdgeSign, an IoT-Enabled Edge–Cloud hybrid system for real-time sign language translation. By expertly combining computer vision, embedded systems, and machine learning, this project creates a useful and real-time gesture recognition and communication system. Capturing hand motions via a camera, the system processes them via the MediaPipe framework for landmark extraction using Raspberry Pi 4 as the core edge device. For gesture categorization, a machine learning model like Random Forest is utilized to guarantee precise and effective recognition. Converting acknowledged gestures into significant text and creating spoken output using a Text-to-Speech (TTS) module helps the system to improve communication even further. Edge–cloud architecture helps to facilitate effective data processing since preliminary gesture recognition is done on the edge device and sophisticated processing such as sentence creation is managed in the cloud. WebSocket or BLE protocols enable real-time transfer of data to a mobile application for interaction and display, thus achieving component communication.

Future enhancement

Though the present system achieves its main goals, several improvements may enhance its performance, usability, and scalability even further. One great advancement is the inclusion of bespoke gesture training, which lets consumers to specify personal gestures and build the system's lexicon. Particularly in diverse backdrops and under different illumination, model accuracy can also be raised by enlarging the dataset size and variety. Future research can concentrate on improving natural language processing abilities by combining more sophisticated artificial intelligence models for better sentence generation and situational understanding. Supporting many languages and local dialects would greatly improve access and usability among several user groups. Furthermore lowering latency and enhancing system responsiveness can come from bettering interaction between edge and cloud elements. Hardware-wise, wearable technologies like smart gloves or sensors can be added to the system for better gesture accuracy and user comfort, therefore improving it. Real-time confirmation to users during gesture input can be provided via feedback systems including vibration or LED signals. On the programming front, cloud-based storage and analytics can support remote updates, ongoing system enhancement, and more efficient usage pattern monitoring.

APPENDIX

A. SAMPLE CODING

```
from aiohttp import web"""
Sign Language Receiver — Laptop
EasyOCR with full preprocessing pipeline:
  flip → grayscale → 3× resize → CLAHE → sharpen → bilateral filter → adaptive threshold → EasyOCR

Pi sends a raw 1280×720 JPEG (no filters).
All image processing happens here on the laptop.
When a word is detected it is pushed back to the Pi as a 'show_word' message
so the connected SH1106 OLED can display it.
"""

from aiohttp import web
import asyncio
import websockets
import json
import numpy as np
import pickle
from collections import deque, Counter
import os
import sys
import aiohttp
import aiohttp_cors
import base64
from datetime import datetime
import traceback
import cv2

# ============================================================
# OPTIONAL IMPORTS
# ============================================================

EASYOCR_AVAILABLE = False
easyocr_reader = None
try:
    import easyocr
    EASYOCR_AVAILABLE = True
    print("[OK] easyocr imported")
except ImportError:
```

```
    print("[WARNING] easyocr not installed — run: pip install easyocr")
except Exception as e:
    print(f"[WARNING] easyocr import error: {e}")

WHISPER_AVAILABLE = False
whisper_model = None
try:
    import whisper
    WHISPER_AVAILABLE = True
    print("[OK] openai-whisper imported")
except ImportError:
    print("[WARNING] openai-whisper not installed — run: pip install openai-whisper")
except Exception as e:
    print(f"[WARNING] whisper import error: {e}")

try:
    import bluetooth
    BLUETOOTH_AVAILABLE = True
except ImportError:
    BLUETOOTH_AVAILABLE = False

GEMINI_AVAILABLE = False
try:
    from google import genai as google_genai
    GEMINI_AVAILABLE = True
    print("[OK] google-genai imported")
except ImportError:
    print("[WARNING] google-genai not installed — run: pip install google-genai")

TTS_AVAILABLE = False
try:
    import pyttsx3
    TTS_AVAILABLE = True
    print("[OK] pyttsx3 imported")
except ImportError:
    print("[WARNING] pyttsx3 not installed — run: pip install pyttsx3")

if sys.platform == 'win32':
    import codecs
    sys.stdout = codecs.getwriter('utf-8')(sys.stdout.buffer, 'strict')
    sys.stderr = codecs.getwriter('utf-8')(sys.stderr.buffer, 'strict')

# ============================================================
```

```
# CONFIGURATION
# ============================================================

BASE_DIR   = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, 'models', 'sign_language_model.pkl')

WEBSOCKET_PORT  = 8765
WEB_SERVER_PORT = 3000

CONFIDENCE_THRESHOLD = 0.70
SMOOTHING_WINDOW     = 7
MIN_CONSENSUS        = 5
MAX_FRAME_SIZE       = 10 * 1024 * 1024  # 10 MB
PRESETS_FILE         = os.path.join(BASE_DIR, 'presets.json')

EASYOCR_MIN_CONFIDENCE = 0.3

GEMINI_API_KEY = "AIzaSyAp8a3uZqkDF-0xog3Flp4D0h2lgUC3koI"
GEMINI_MODEL   = "gemini-2.5-flash"

GEMINI_PROMPT_NO_CONTEXT = """\
You are a sentence builder for a sign language recognition system.
The user will give you a list of words detected from sign language.
Your job is to form ONE single, complete, grammatically correct, natural-sounding sentence
using ALL of those words. You have full creative freedom to add connecting words, articles,
prepositions, verb tenses, or any other words needed to make the sentence sound fluent and natural.
Every word the user gives you MUST appear in the final sentence — do not drop any.
Reply with ONLY the sentence. No explanation, no punctuation outside the sentence, no quotes.\
"""

GEMINI_PROMPT_WITH_CONTEXT = """\
You are a sentence builder for a sign language recognition system.
The user will give you:
  1. A TOPIC / CONTEXT — this is the situation or subject being discussed.
  2. A list of WORDS detected from sign language.

Your job is to form ONE single, complete, grammatically correct, natural-sounding sentence
that fits within the given topic/context AND uses ALL of the provided words.
You have full creative freedom to add any connecting words needed to make it fluent.
Every provided word MUST appear in the final sentence — do not drop any.
Reply with ONLY the sentence. No explanation, no punctuation outside the sentence, no quotes.\
"""
```

```
TTS_RATE = 160

KEYPAD_ACTION_LABELS = {
   'scroll_down':    'Down Arrow',
   'preset_1':       'Preset Slot 1',
   'preset_2':       'Preset Slot 2',
   'add_word':       'Add Word',
   'preset_3':       'Preset Slot 3',
   'standard_key_5': 'Standard key input',
   'standard_key_6': 'Standard key input',
   'send_to_ai':     'Gemini',
   'standard_key_7': 'Standard key input',
   'standard_key_8': 'Standard key input',
   'standard_key_9': 'Standard key input',
   'request_ocr':    'OCR',
   'delete_last':    'Delete',
   'scroll_up':      'Up Arrow',
   'clear_words':    'Clear',
   'toggle_audio':   'Record Toggle',
}

# ============================================================
# GLOBAL STATE
# ============================================================

class SystemState:
   def __init__(self):
      self.model         = None
      self.scaler        = None
      self.label_encoder = None
      self.actions       = []

      self.prediction_history = deque(maxlen=SMOOTHING_WINDOW)
      self.landmark_buffer    = deque(maxlen=5)
      self.last_prediction    = None
      self.prediction_count   = 0

      self.words   = []
      self.context = ""

      self.pi_websocket    = None
      self.web_clients     = set()
      self.pi_connected    = False
```

```
        self.last_word       = ""
        self.last_confidence = 0.0

        self.last_frame_base64 = None
        self.last_audio_base64 = None
        self.last_ocr_result   = None
        self.last_sentence     = ""
        self.last_dev_frame    = None
        self.dev_stream_enabled = False
        self.presets = {
            1: "I want a coffee",
            2: "Call me later",
            3: "Help me",
        }
        self.last_keypad_event = None

        self.error_log = deque(maxlen=50)
        self.performance_stats = {
            'frames_received':     0,
            'predictions_made':    0,
            'avg_processing_time': 0,
            'ocr_calls':           0,
            'audio_calls':         0,
            'gemini_calls':        0,
        }

state = SystemState()


def _load_presets():
    if not os.path.exists(PRESETS_FILE):
        return
    try:
        with open(PRESETS_FILE, 'r', encoding='utf-8') as f:
            raw = json.load(f)
        for slot in (1, 2, 3):
            value = str(raw.get(str(slot), raw.get(slot, state.presets[slot]))).strip()
            state.presets[slot] = value
        print(f"[PRESET] Loaded presets from {PRESETS_FILE}")
    except Exception as e:
        log_error("Loading presets", e, "Check presets.json format")
```

```
def _save_presets():
    try:
        with open(PRESETS_FILE, 'w', encoding='utf-8') as f:
            json.dump({str(k): v for k, v in state.presets.items()}, f, indent=2, ensure_ascii=False)
    except Exception as e:
        log_error("Saving presets", e, "Check file permissions for presets.json")

# ============================================================
# ERROR HELPERS
# ============================================================

def log_error(context, error, solution=""):
    entry = {
        'timestamp': datetime.now().isoformat(),
        'context':   context,
        'error':     str(error),
        'type':      type(error).__name__,
        'solution':  solution,
        'traceback': traceback.format_exc(),
    }
    state.error_log.append(entry)
    print(f"\n[ERROR] {context}")
    print(f"  Type:    {type(error).__name__}")
    print(f"  Message: {error}")
    if solution:
        print(f"  Fix:     {solution}")
    return entry

async def broadcast_error(context, error, solution=""):
    entry = log_error(context, error, solution)
    await broadcast_to_web_clients({'type': 'error', 'error': entry})

# ============================================================
# MODEL LOADING
# ============================================================

def load_model():
    candidates = [
        MODEL_PATH,
        os.path.join('models', 'sign_language_model.pkl'),
        'sign_language_model.pkl',
    ]
    for path in candidates:
```

```
        if not os.path.exists(path):
            continue
        try:
            with open(path, 'rb') as f:
                data = pickle.load(f)
            state.model         = data['model']
            state.scaler        = data['scaler']
            state.label_encoder = data['label_encoder']
            state.actions       = data['actions']
            print(f"[OK] Model loaded: {path}")
            print(f"[OK] Actions: {state.actions}")
            return True
        except Exception as e:
            log_error(f"Loading model: {path}", e, "Try retraining.")
    log_error("Model not found", FileNotFoundError(), "Run: python train_model.py")
    return False

# ============================================================
# LANDMARK PROCESSING
# ============================================================

def normalize_landmarks(arr):
    try:
        res  = arr.copy() - arr[0]
        dist = np.max(np.linalg.norm(res, axis=1))
        if dist > 0:
            res /= dist
        mid   = res[9]
        angle = np.arctan2(mid[1], mid[0])
        c, s  = np.cos(-angle), np.sin(-angle)
        res[:, :2] = res[:, :2] @ np.array([[c, -s], [s, c]]).T
        return res.flatten()
    except Exception as e:
        log_error("Landmark normalization", e)
        return None

def get_stabilized_landmarks(arr):
    n = normalize_landmarks(arr)
    if n is None:
        return None
    state.landmark_buffer.append(n)
    return np.mean(state.landmark_buffer, axis=0) if len(state.landmark_buffer) >= 3 else n
```

```
def preprocess_landmarks(raw):
    try:
        raw  = np.array(raw, dtype=np.float32)
        hand = raw[0:63].reshape(21, 3)
        if np.all(hand == -1):
            hand2 = raw[64:127].reshape(21, 3)
            if np.all(hand2 == -1):
                return None
            hand = hand2
        return get_stabilized_landmarks(hand)
    except Exception as e:
        log_error("Landmark preprocessing", e)
        return None

# ============================================================
# PREDICTION
# ============================================================

def predict_sign(landmarks):
    if state.model is None:
        return None, 0.0, []
    try:
        features = preprocess_landmarks(landmarks)
        if features is None:
            return None, 0.0, []
        features = state.scaler.transform(features.reshape(1, -1))
        probs    = state.model.predict_proba(features)[0]
        idx      = np.argmax(probs)
        word     = state.label_encoder.classes_[idx]
        conf     = float(probs[idx])
        top3     = [(state.label_encoder.classes_[i], float(probs[i]))
                    for i in np.argsort(probs)[-3:][::-1]]
        state.prediction_count += 1
        state.performance_stats['predictions_made'] += 1
        return word, conf, top3
    except Exception as e:
        log_error("Prediction", e)
        return None, 0.0, []

def smooth_prediction(word, confidence):
    if confidence < CONFIDENCE_THRESHOLD:
        return None, False
    state.prediction_history.append(word)
```

```
    if len(state.prediction_history) < MIN_CONSENSUS:
        return None, False
    common, count = Counter(state.prediction_history).most_common(1)[0]
    if count >= MIN_CONSENSUS and common != state.last_prediction:
        state.last_prediction = common
        return common, True
    return None, False

# ============================================================
# EASYOCR
# ============================================================

def _preprocess_image_for_ocr(img_bgr):
    img_bgr = cv2.flip(img_bgr, 1)
    img_bgr = cv2.resize(img_bgr, None, fx=3, fy=3, interpolation=cv2.INTER_CUBIC)
    return img_bgr

def _run_easyocr(img_bgr):
    if not EASYOCR_AVAILABLE or easyocr_reader is None:
        raise RuntimeError("EasyOCR not initialised — run: pip install easyocr")
    processed  = _preprocess_image_for_ocr(img_bgr)
    results    = easyocr_reader.readtext(processed)
    detections = []
    text_parts = []
    for bbox, text, prob in results:
        print(f"  [easyocr] {prob:.2f}  {text!r}")
        if prob >= EASYOCR_MIN_CONFIDENCE and text.strip():
            text_parts.append(text.strip())
            detections.append({
                'text':       text.strip(),
                'confidence': round(float(prob), 3),
                'bbox':       [[int(p[0]), int(p[1])] for p in bbox],
            })
        else:
            print(f"  [skip]    {prob:.2f}  {text!r}")
    return ' '.join(text_parts), detections

def _encode_processed_preview(img_bgr):
    try:
        processed = _preprocess_image_for_ocr(img_bgr)
        preview = cv2.resize(processed, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
        _, buf = cv2.imencode('.jpg', preview, [cv2.IMWRITE_JPEG_QUALITY, 80])
        return base64.b64encode(buf.tobytes()).decode('utf-8')
```

```
    except Exception:
        return None

# ============================================================
# GEMINI
# ============================================================

def _call_gemini(words: list, context: str) -> str:
    if not GEMINI_AVAILABLE:
        raise RuntimeError("google-genai not installed — run: pip install google-genai")
    client    = google_genai.Client(api_key=GEMINI_API_KEY)
    words_str = ', '.join(words)
    if context.strip():
        prompt = (
            f"{GEMINI_PROMPT_WITH_CONTEXT}\n\n"
            f"TOPIC / CONTEXT: {context.strip()}\n\n"
            f"WORDS: {words_str}"
        )
    else:
        prompt = f"{GEMINI_PROMPT_NO_CONTEXT}\n\nWORDS: {words_str}"
    print(f"[GEMINI] Sending → words={words_str!r}  context={context.strip()!r}")
    response = client.models.generate_content(model=GEMINI_MODEL, contents=prompt)
    sentence = response.text.strip().strip('"').strip("'")
    print(f"[GEMINI] Response: {sentence!r}")
    return sentence

# ============================================================
# TTS
# ============================================================

def _speak(text: str):
    if not TTS_AVAILABLE:
        raise RuntimeError("pyttsx3 not installed — run: pip install pyttsx3")
    engine = pyttsx3.init()
    engine.setProperty('rate',   TTS_RATE)
    engine.setProperty('volume', 1.0)
    engine.say(text)
    engine.runAndWait()
    engine.stop()
    print(f"[TTS] Spoke: {text!r}")

# ============================================================
# BROADCAST  — goes to ALL web clients AND the Pi OLED
```

```
# ============================================================

async def broadcast_to_web_clients(message):
    """
    Send to every browser client AND forward to Pi so the OLED stays in sync.
    'show_word' is excluded here because send_word_to_web() handles it directly.
    """
    # ── Browser clients ──────────────────────────────────────────────────────
    dead = set()
    for client in state.web_clients:
        try:
            await client.send_str(json.dumps(message))
        except Exception:
            dead.add(client)
    state.web_clients -= dead

    # ── Pi / OLED — forward everything except show_word ──────────────
    if state.pi_websocket and message.get('type') != 'show_word':
        try:
            await state.pi_websocket.send(json.dumps(message))
        except Exception:
            pass


async def send_word_to_web(word, confidence):
    """Broadcast detected word to browser clients AND push show_word to Pi OLED."""
    await broadcast_to_web_clients({
        'type':       'word_detected',
        'word':       word,
        'confidence': confidence,
        'timestamp':  datetime.now().isoformat(),
    })
    # Direct show_word push to Pi for the OLED prediction strip
    if state.pi_websocket:
        try:
            await state.pi_websocket.send(json.dumps({
                'type':       'show_word',
                'word':       word,
                'confidence': confidence,
            }))
        except Exception:
            pass
```

```
async def send_status_to_web():
    await broadcast_to_web_clients({
        'type':           'status',
        'pi_connected':   state.pi_connected,
        'words_count':    len(state.words),
        'last_word':      state.last_word,
        'last_confidence': state.last_confidence,
        'stats':          state.performance_stats,
    })


async def send_presets_to_pi():
    if not state.pi_websocket:
        return
    try:
        await state.pi_websocket.send(json.dumps({
            'type': 'presets_updated',
            'presets': {str(slot): text for slot, text in state.presets.items()},
        }))
    except Exception as e:
        log_error("Send presets to Pi", e)

# ============================================================
# PI WEBSOCKET HANDLER
# ============================================================

async def handle_pi_connection(websocket):
    state.pi_websocket = websocket
    state.pi_connected = True
    print("[PI] Connected!")
    await send_presets_to_pi()
    await send_status_to_web()

    try:
        async for message in websocket:
            try:
                state.performance_stats['frames_received'] += 1
                data     = json.loads(message)
                msg_type = data.get('type') if isinstance(data, dict) else None

                if msg_type == 'ping':
                    continue
```

```
elif msg_type == 'frame':
    await handle_frame_from_pi(data)
    continue
elif msg_type == 'audio':
    await handle_audio_from_pi(data)
    continue
elif msg_type == 'audio_status':
    await broadcast_to_web_clients(data)
    continue
elif msg_type == 'dev_frame':
    frame_b64 = data.get('frame')
    if frame_b64:
        state.last_dev_frame = frame_b64
        await broadcast_to_web_clients({
            'type': 'dev_frame',
            'image': frame_b64,
            'timestamp': data.get('timestamp'),
        })
    continue
elif msg_type == 'dev_stream_status':
    state.dev_stream_enabled = bool(data.get('enabled', False))
    await broadcast_to_web_clients({
        'type': 'dev_stream_status',
        'enabled': state.dev_stream_enabled,
    })
    continue
elif msg_type == 'keypad_press':
    state.last_keypad_event = {
        'key': data.get('key', ''),
        'action': data.get('action', ''),
        'timestamp': datetime.now().isoformat(),
    }
    await broadcast_to_web_clients({
        'type': 'keypad_press',
        'key': data.get('key', ''),
        'action': data.get('action', ''),
        'action_label': KEYPAD_ACTION_LABELS.get(data.get('action', ''), data.get('action', '')),
    })
    continue

if isinstance(data, dict) and data.get('command'):
    await handle_web_command(data, None, source='pi')
    continue
```

```
            # ── Landmark / sign data ──────────────────────────────
            landmarks = data.get('landmarks') if isinstance(data, dict) else data
            if landmarks is None:
                continue
            landmarks = np.array(landmarks, dtype=np.float32)
            if landmarks.shape[0] != 128:
                continue
            word, conf, _ = predict_sign(landmarks)
            if word is None:
                continue
            final, show = smooth_prediction(word, conf)
            if show:
                state.last_word       = final
                state.last_confidence = conf
                print(f"[DETECTED] {final.upper()} ({conf:.1%})")
                await send_word_to_web(final, conf)

        except json.JSONDecodeError as e:
            await broadcast_error("JSON parse from Pi", e, "Pi must send valid JSON")
        except Exception as e:
            await broadcast_error("Processing Pi message", e)

    except websockets.exceptions.ConnectionClosed as e:
        print(f"[PI] Connection closed: {getattr(e, 'reason', 'unknown')}")
    except Exception as e:
        await broadcast_error("Pi connection handler", e)
    finally:
        state.pi_connected = False
        state.pi_websocket = None
        print("[PI] Disconnected")
        await send_status_to_web()

# ============================================================
# OCR FRAME HANDLER
# ============================================================

async def handle_frame_from_pi(data):
    try:
        state.performance_stats['ocr_calls'] += 1

        if not EASYOCR_AVAILABLE or easyocr_reader is None:
            raise RuntimeError("EasyOCR not initialised — run: pip install easyocr")
```

```
frame_b64 = data.get('frame', '')
if not frame_b64:
    raise ValueError("Pi sent an empty 'frame' field")

clean = frame_b64.strip().replace('\n', '').replace('\r', '').replace(' ', '')
if not clean:
    raise ValueError("base64 string is empty")

try:
    img_bytes = base64.b64decode(clean + '==')
except Exception as e:
    raise ValueError(f"base64 decode failed: {e}")

if len(img_bytes) < 500:
    raise ValueError(f"Decoded frame only {len(img_bytes)} bytes")

print(f"[OCR] Frame received: {len(img_bytes) // 1024} KB")
state.last_frame_base64 = frame_b64

arr     = np.frombuffer(img_bytes, dtype=np.uint8)
img_bgr = cv2.imdecode(arr, cv2.IMREAD_COLOR)
if img_bgr is None:
    raise ValueError("cv2.imdecode failed — not a valid JPEG")

h, w = img_bgr.shape[:2]
print(f"[OCR] Image decoded: {w}×{h} → EasyOCR ...")

loop = asyncio.get_event_loop()
try:
    text, detections = await asyncio.wait_for(
        loop.run_in_executor(None, _run_easyocr, img_bgr),
        timeout=60.0
    )
except asyncio.TimeoutError:
    raise RuntimeError("OCR timed out after 60 s")

print(f"[OCR] Extracted ({len(text)} chars): {text[:200]!r}")

processed_preview_b64 = await loop.run_in_executor(
    None, _encode_processed_preview, img_bgr
)
```

```
        state.last_ocr_result = {
            'text':       text,
            'detections': detections,
            'timestamp':  datetime.now().isoformat(),
        }

        if text:
            await broadcast_to_web_clients({
                'type':              'ocr_result',
                'text':              text,
                'detections':        detections,
                'image_preview':     frame_b64,
                'processed_preview':  processed_preview_b64,
                'needs_confirmation': True,
            })
        else:
            await broadcast_to_web_clients({
                'type': 'error',
                'error': {
                    'context':  'OCR',
                    'error':    'No text detected in image',
                    'solution': 'Ensure text is clear, well-lit and camera is steady.',
                },
            })

    except ValueError as e:
        await broadcast_error("OCR - bad image data", e, "Check Pi is sending a valid base64 JPEG")
    except RuntimeError as e:
        await broadcast_error("OCR - engine error", e, "run: pip install easyocr")
    except Exception as e:
        await broadcast_error("OCR processing", e, "Check easyocr installation")

# ============================================================
# AUDIO HANDLER
# ============================================================

def _transcribe_with_whisper(audio_bytes, rate, channels):
    audio_np = np.frombuffer(audio_bytes, dtype=np.int16).astype(np.float32) / 32768.0
    if channels == 2:
        audio_np = audio_np.reshape(-1, 2).mean(axis=1)
    if rate != 16000:
        target_len = int(len(audio_np) * 16000 / rate)
        audio_np   = np.interp(
```

```
            np.linspace(0, len(audio_np) - 1, target_len),
            np.arange(len(audio_np)),
            audio_np
        ).astype(np.float32)
    print(f"[AUDIO] Running Whisper on {len(audio_np)/16000:.1f}s ...")
    result = whisper_model.transcribe(audio_np, fp16=False, language='en')
    return result.get('text', '').strip()

async def handle_audio_from_pi(data):
    try:
        state.performance_stats['audio_calls'] += 1

        if not WHISPER_AVAILABLE or whisper_model is None:
            raise RuntimeError("Whisper not initialised — run: pip install openai-whisper")

        audio_b64 = data.get('audio', '')
        duration  = data.get('duration', 0)
        rate      = data.get('rate', 16000)
        channels  = data.get('channels', 1)

        if not audio_b64:
            raise ValueError("No audio data received from Pi")

        state.last_audio_base64 = audio_b64
        print(f"[AUDIO] Received {duration:.1f}s at {rate} Hz ({channels}ch)")

        audio_bytes = base64.b64decode(audio_b64)

        await broadcast_to_web_clients({
            'type':     'audio_status',
            'status':   'transcribing',
            'duration': duration,
        })

        loop = asyncio.get_event_loop()
        try:
            text = await asyncio.wait_for(
                loop.run_in_executor(
                    None, _transcribe_with_whisper, audio_bytes, rate, channels
                ),
                timeout=120.0
            )
        except asyncio.TimeoutError:
```

```
            raise RuntimeError("Whisper transcription timed out after 120 s")

        print(f"[AUDIO] Transcribed: {text!r}")

        if text:
            await broadcast_to_web_clients({
                'type':             'audio_result',
                'text':             text,
                'duration':         duration,
                'needs_confirmation': True,
            })
        else:
            await broadcast_to_web_clients({
                'type': 'error',
                'error': {
                    'context':  'Audio',
                    'error':    'Whisper returned empty transcription',
                    'solution': 'Speak clearly — check microphone and try again.',
                },
            })

    except ValueError as e:
        await broadcast_error("Audio - bad data", e, "Check Pi is sending valid audio")
    except RuntimeError as e:
        await broadcast_error("Audio - engine error", e, "run: pip install openai-whisper")
    except Exception as e:
        await broadcast_error("Audio processing", e)

# ============================================================
# SEND COMMAND TO PI
# ============================================================

async def send_command_to_pi(command, params=None):
    if not state.pi_websocket:
        return {'success': False, 'error': 'Pi not connected'}
    try:
        await state.pi_websocket.send(json.dumps({
            'type':      'command',
            'command':   command,
            'params':    params or {},
            'timestamp': datetime.now().isoformat(),
        }))
        print(f"[CMD] -> Pi: {command}")
```

```
        return {'success': True}
    except Exception as e:
        log_error("Send command to Pi", e)
        return {'success': False, 'error': str(e)}

# ============================================================
# BROWSER AUDIO HANDLER
# ============================================================

async def handle_browser_audio(data, ws):
    try:
        state.performance_stats['audio_calls'] += 1

        if not WHISPER_AVAILABLE or whisper_model is None:
            raise RuntimeError("Whisper not loaded — run: pip install openai-whisper")

        audio_b64 = data.get('audio', '')
        duration  = data.get('duration', 0)

        if not audio_b64:
            raise ValueError("No audio data in browser_audio command")

        await broadcast_to_web_clients({
            'type':     'audio_status',
            'status':   'transcribing',
            'duration': duration,
        })

        audio_bytes = base64.b64decode(audio_b64)
        audio_np    = np.frombuffer(audio_bytes, dtype=np.float32).copy()
        audio_np    = np.clip(audio_np, -1.0, 1.0)

        if len(audio_np) < 1600:
            raise ValueError("Audio too short — hold the button longer")

        loop = asyncio.get_event_loop()
        try:
            text = await asyncio.wait_for(
                loop.run_in_executor(None, _transcribe_numpy, audio_np),
                timeout=120.0
            )
        except asyncio.TimeoutError:
            raise RuntimeError("Whisper timed out after 120 s")
```

```
        print(f"[BROWSER AUDIO] Transcribed: {text!r}")

        if text:
            await broadcast_to_web_clients({
                'type':             'audio_result',
                'text':             text,
                'duration':          duration,
                'source':           'browser',
                'needs_confirmation': True,
            })
        else:
            await broadcast_to_web_clients({
                'type': 'error',
                'error': {
                    'context':  'Browser Audio',
                    'error':    'Whisper returned empty result',
                    'solution': 'Speak clearly and close to the microphone.',
                },
            })

    except ValueError as e:
        await broadcast_to_web_clients({'type': 'error', 'error': {
            'context': 'Browser Audio', 'error': str(e), 'solution': ''}})
    except RuntimeError as e:
        await broadcast_to_web_clients({'type': 'error', 'error': {
            'context': 'Browser Audio', 'error': str(e),
            'solution': 'pip install openai-whisper'}})
    except Exception as e:
        log_error("Browser audio", e)
        await broadcast_to_web_clients({'type': 'error', 'error': {
            'context': 'Browser Audio', 'error': str(e), 'solution': ''}})


def _transcribe_numpy(audio_np):
    result = whisper_model.transcribe(audio_np, fp16=False, language='en')
    return result.get('text', '').strip()

# ============================================================
# WEB SERVER
# ============================================================

async def websocket_handler(request):
```

```
    ws = web.WebSocketResponse()
    await ws.prepare(request)
    state.web_clients.add(ws)
    print(f"[WEB] Client connected ({len(state.web_clients)} total)")

    await ws.send_str(json.dumps({
        'type':         'init',
        'pi_connected': state.pi_connected,
        'words':        state.words,
        'context':      state.context,
        'actions':      state.actions,
        'presets':      {str(slot): text for slot, text in state.presets.items()},
        'keypad_map':   KEYPAD_ACTION_LABELS,
        'last_keypad_event': state.last_keypad_event,
        'dev_stream_enabled': state.dev_stream_enabled,
        'stats':        state.performance_stats,
        'features': {
            'ocr':       EASYOCR_AVAILABLE,
            'audio':     WHISPER_AVAILABLE,
            'bluetooth': BLUETOOTH_AVAILABLE,
            'gemini':    GEMINI_AVAILABLE,
            'tts':       TTS_AVAILABLE,
        },
    }))

    try:
        async for msg in ws:
            if msg.type == aiohttp.WSMsgType.TEXT:
                try:
                    await handle_web_command(json.loads(msg.data), ws)
                except Exception as e:
                    await broadcast_error("Web command", e)
            elif msg.type == aiohttp.WSMsgType.ERROR:
                print(f"[WEB] Error: {ws.exception()}")
    finally:
        state.web_clients.discard(ws)
        print(f"[WEB] Client disconnected ({len(state.web_clients)} remaining)")
    return ws


async def _send_ws_if_present(ws, payload):
    if ws is None:
        return
```

```
    await ws.send_str(json.dumps(payload))


async def handle_web_command(data, ws, source='web'):
    cmd = data.get('command')
    print(f"[CMD:{source.upper()}] {cmd}")

    # ── Preset Trigger (from Pi keypad) ──────────────────────
    if cmd == 'trigger_preset':
        try:
            slot = int(data.get('slot', 0) or 0)
        except (TypeError, ValueError):
            slot = 0
        sentence = str(data.get('sentence', '')).strip()
        if not sentence and slot in state.presets:
            sentence = state.presets[slot]
        if not sentence:
            await broadcast_to_web_clients({'type': 'error', 'error': {
                'context': 'Preset',
                'error': f'Preset slot {slot} is empty',
                'solution': 'Set preset text in the UI and save it',
            }})
            return
        state.words.append(sentence)
        await broadcast_to_web_clients({
            'type': 'preset_triggered',
            'slot': slot,
            'sentence': sentence,
            'source': source,
        })
        await broadcast_to_web_clients({'type': 'words_updated', 'words': state.words})
        return

    # ── Standard Key Input (5/6/7/8/9) ──────────────────────
    if cmd == 'standard_key_input':
        key = str(data.get('key', '')).strip()
        if key:
            state.words.append(key)
            await broadcast_to_web_clients({
                'type': 'standard_key_input',
                'key': key,
                'source': source,
            })
```

```
            await broadcast_to_web_clients({'type': 'words_updated', 'words': state.words})
        return

    # ── Preset Update (from Web UI) ──────────────────────────────────
    if cmd == 'update_preset':
        try:
            slot = int(data.get('slot', 0) or 0)
        except (TypeError, ValueError):
            slot = 0
        text = str(data.get('text', '')).strip()
        if slot not in (1, 2, 3):
            await broadcast_to_web_clients({'type': 'error', 'error': {
                'context': 'Preset',
                'error': f'Invalid preset slot: {slot}',
                'solution': 'Use only preset slots 1, 2 or 3',
            }})
            return
        state.presets[slot] = text
        _save_presets()
        await broadcast_to_web_clients({
            'type': 'presets_updated',
            'presets': {str(k): v for k, v in state.presets.items()},
            'updated_slot': slot,
        })
        await send_presets_to_pi()
        return

    # ── Add Word ─────────────────────────────────────────────────────
    # Accepts an explicit 'word' field (from Pi keypad) OR falls
    # back to state.last_word (from web UI Add Word button).
    if cmd == 'add_word':
        word = data.get('word', '').strip()
        if not word:
            word = state.last_word.strip()
        if word:
            state.words.append(word)
            print(f"[ADD WORD] '{word}' → words now: {state.words}")
            await broadcast_to_web_clients({'type': 'words_updated', 'words': state.words})
        else:
            # Nothing to add — tell the client
            await broadcast_to_web_clients({'type': 'error', 'error': {
                'context':  'Add Word',
                'error':    'No word detected yet — sign a word first',
```

```
            'solution': 'Make a sign and wait for the prediction to appear',
        }})
    return

# ── Delete Last ──────────────────────────────────────────────────────
elif cmd == 'delete_last':
    if state.words:
        removed = state.words.pop()
        print(f"[DELETE] Removed '{removed}' → words now: {state.words}")
    await broadcast_to_web_clients({'type': 'words_updated', 'words': state.words})

# ── Clear All ────────────────────────────────────────────────────────
elif cmd == 'clear_words':
    state.words = []
    print("[CLEAR] All words cleared")
    await broadcast_to_web_clients({'type': 'words_updated', 'words': state.words})

# ── Send to Gemini ───────────────────────────────────────────────────
elif cmd == 'send_to_ai':
    if not state.words:
        await broadcast_to_web_clients({'type': 'error', 'error': {
            'context':  'Gemini',
            'error':    'No words to send — add some words first',
            'solution': 'Sign some words and add them before sending to AI',
        }})
        return

    if not GEMINI_AVAILABLE:
        await broadcast_to_web_clients({'type': 'error', 'error': {
            'context':  'Gemini',
            'error':    'google-generativeai not installed',
            'solution': 'pip install google-generativeai',
        }})
        return

    # Notify ALL clients (browser + Pi OLED) that Gemini is generating
    await broadcast_to_web_clients({'type': 'gemini_status', 'status': 'generating'})

    state.performance_stats['gemini_calls'] += 1
    words_snapshot   = list(state.words)
    context_snapshot = state.context

    loop = asyncio.get_event_loop()
```

```
    try:
        sentence = await asyncio.wait_for(
            loop.run_in_executor(
                None, _call_gemini, words_snapshot, context_snapshot
            ),
            timeout=30.0
        )
    except asyncio.TimeoutError:
        await broadcast_to_web_clients({'type': 'error', 'error': {
            'context':  'Gemini',
            'error':    'Request timed out after 30 s',
            'solution': 'Check your internet connection and API key',
        }})
        await broadcast_to_web_clients({'type': 'gemini_status', 'status': 'idle'})
        return
    except Exception as e:
        log_error("Gemini call", e, "Check GEMINI_API_KEY in laptop_receiver.py")
        await broadcast_to_web_clients({'type': 'error', 'error': {
            'context':  'Gemini',
            'error':    str(e),
            'solution': 'Check GEMINI_API_KEY in laptop_receiver.py',
        }})
        await broadcast_to_web_clients({'type': 'gemini_status', 'status': 'idle'})
        return

    state.last_sentence = sentence

    # Broadcast sentence to ALL clients (browser + Pi OLED)
    await broadcast_to_web_clients({
        'type':     'gemini_sentence',
        'sentence': sentence,
        'words':    words_snapshot,
        'context':  context_snapshot,
    })

    if TTS_AVAILABLE:
        loop.run_in_executor(None, _speak, sentence)
    else:
        print("[TTS] Skipped — pyttsx3 not installed")

# ── Speak Sentence ──────────────────────────────────────────────────────
elif cmd == 'speak_sentence':
    text = data.get('sentence', state.last_sentence)
```

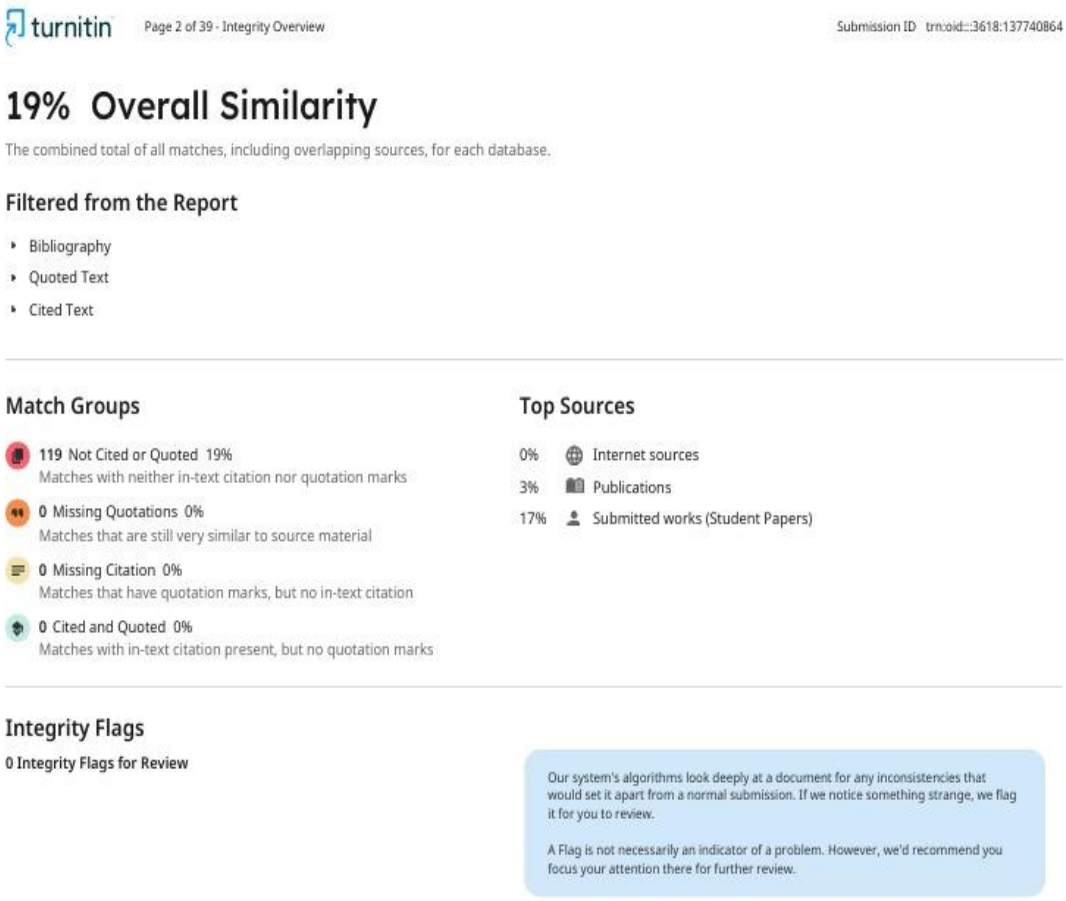
```
    if text and TTS_AVAILABLE:
        loop = asyncio.get_event_loop()
        loop.run_in_executor(None, _speak, text)
    elif not TTS_AVAILABLE:
        await broadcast_to_web_clients({'type': 'error', 'error': {
            'context': 'TTS', 'error': 'pyttsx3 not installed',
            'solution': 'pip install pyttsx3'}})

# ── Context ──────────────────────────────────────────────────────────
elif cmd == 'remove_context':
    state.context = ""
    await broadcast_to_web_clients({'type': 'context_updated', 'context': state.context})

elif cmd == 'add_context':
    state.context = (state.context + ' ' + data.get('text', '')).strip()
    await broadcast_to_web_clients({'type': 'context_updated', 'context': state.context})

# ── OCR ──────────────────────────────────────────────────────────────
elif cmd == 'request_ocr':
    result = await send_command_to_pi('capture_frame')
    await _send_ws_if_present(ws, result)

# ── Audio ────────────────────────────────────────────────────────────
elif cmd == 'browser_audio':
    await handle_browser_audio(data, ws)

elif cmd == 'request_audio':
    duration = data.get('duration', 10)
    result = await send_command_to_pi('record_audio', {'duration': duration})
    await _send_ws_if_present(ws, result)

elif cmd == 'start_audio':
    result = await send_command_to_pi('start_audio')
    await _send_ws_if_present(ws, result)

elif cmd == 'stop_audio':
    result = await send_command_to_pi('stop_audio')
    await _send_ws_if_present(ws, result)

elif cmd == 'toggle_dev_stream':
    enabled = bool(data.get('enabled', False))
    result = await send_command_to_pi('toggle_dev_stream', {'enabled': enabled})
    await _send_ws_if_present(ws, result)
```

```
        if result.get('success'):
            state.dev_stream_enabled = enabled
            await broadcast_to_web_clients({'type': 'dev_stream_status', 'enabled': enabled})
        else:
            state.dev_stream_enabled = False
            await broadcast_to_web_clients({'type': 'dev_stream_status', 'enabled': False})
            await broadcast_to_web_clients({'type': 'error', 'error': {
                'context': 'Dev Stream',
                'error': result.get('error', 'Pi not connected'),
                'solution': 'Start pi_sender.py and ensure Pi websocket is connected',
            }})

    # ── Bluetooth ──────────────────────────────────────────────────────────
    elif cmd == 'scan_bluetooth':
        devices = []
        if BLUETOOTH_AVAILABLE:
            try:
                raw     = bluetooth.discover_devices(duration=5, lookup_names=True)
                devices = [{'address': a, 'name': n} for a, n in raw]
            except Exception as e:
                log_error("Bluetooth scan", e)
        await _send_ws_if_present(ws, {'type': 'bluetooth_devices', 'devices': devices})

    # ── Debug ──────────────────────────────────────────────────────────────
    elif cmd == 'get_debug_data':
        await _send_ws_if_present(ws, {
            'type':       'debug_data',
            'last_frame': state.last_frame_base64,
            'last_dev_frame': state.last_dev_frame,
            'last_audio': state.last_audio_base64,
            'last_ocr':   state.last_ocr_result,
            'dev_stream_enabled': state.dev_stream_enabled,
            'error_log':  list(state.error_log)[-10:],
            'stats':      state.performance_stats,
        })


async def serve_index(request):
    path = os.path.join(BASE_DIR, 'web', 'index.html')
    return web.FileResponse(path) if os.path.exists(path) else \
           web.Response(text="Web interface not found", status=404)

# ═══════════════════════════════════════════════════════════════
```

```
# MAIN
# ============================================================

async def main():
    print("\n" + "=" * 60)
    print("SIGN LANGUAGE RECOGNITION SYSTEM")
    print("=" * 60)

    if not load_model():
        print("\n[FATAL] No model — run: python train_model.py")
        return

    _load_presets()

    print(f"\n[CONFIG]")
    print(f"  WebSocket port : {WEBSOCKET_PORT}")
    print(f"  Web server port: {WEB_SERVER_PORT}")
    print(f"  EasyOCR        : {'available' if EASYOCR_AVAILABLE else 'not available'}")
    print(f"  Whisper        : {'available' if WHISPER_AVAILABLE else 'not available'}")
    print(f"  Gemini         : {'available' if GEMINI_AVAILABLE else 'not available'}")
    print(f"  TTS (pyttsx3)  : {'available' if TTS_AVAILABLE else 'not available'}")

    global easyocr_reader
    if EASYOCR_AVAILABLE:
        try:
            print("[OCR] Loading EasyOCR model ...")
            easyocr_reader = easyocr.Reader(['en'], gpu=False)
            dummy = np.zeros((64, 64, 3), dtype=np.uint8)
            easyocr_reader.readtext(dummy)
            print("[OCR] EasyOCR ready ✓")
        except Exception as e:
            print(f"[OCR] WARNING: {e}")
            easyocr_reader = None

    global whisper_model
    if WHISPER_AVAILABLE:
        try:
            print("[AUDIO] Loading Whisper 'base' model ...")
            whisper_model = whisper.load_model('base')
            print("[AUDIO] Whisper ready ✓")
        except Exception as e:
            print(f"[AUDIO] WARNING: {e}")
            whisper_model = None
```

```
    pi_server = await websockets.serve(
        handle_pi_connection, "0.0.0.0", WEBSOCKET_PORT,
        ping_interval=20, ping_timeout=20,
        max_size=MAX_FRAME_SIZE,
    )
    print(f"[OK] Pi WebSocket on port {WEBSOCKET_PORT}")

    app  = web.Application()
    cors = aiohttp_cors.setup(app, defaults={
        "*": aiohttp_cors.ResourceOptions(
            allow_credentials=True, expose_headers="*", allow_headers="*")
    })
    app.router.add_get('/', serve_index)
    app.router.add_get('/ws', websocket_handler)
    app.router.add_static('/static', os.path.join(BASE_DIR, 'web', 'static'), name='static')
    for route in list(app.router.routes()):
        cors.add(route)

    runner = web.AppRunner(app)
    await runner.setup()
    await web.TCPSite(runner, '0.0.0.0', WEB_SERVER_PORT).start()

    print(f"[OK] Web server on http://localhost:{WEB_SERVER_PORT}")
    print("=" * 60)
    print(f"  EasyOCR : {'✓ loaded' if easyocr_reader else '✗ not loaded'}")
    print(f"  Whisper : {'✓ loaded' if whisper_model else '✗ not loaded'}")
    print(f"  Gemini  : {'✓ ready' if GEMINI_AVAILABLE else '✗ not installed'}")
    print(f"  TTS     : {'✓ ready' if TTS_AVAILABLE else '✗ not installed'}")
    print("Open browser and start Pi sender")
    print("=" * 60 + "\n")

    await asyncio.Future()


if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("\n[SHUTDOWN] Stopped")
    except Exception as e:
        log_error("Main", e)
```

B. PLAGIARISM REPORT



turnitin Page 2 of 39 - Integrity Overview Submission ID trn:oid:::3618:137740864

19% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text

Match Groups

119 Not Cited or Quoted 19%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
3% Publications
17% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.